

# **Predictive Maintenance in Heavy Machinery: A Machine Learning Approach Using Vibration and Temperature Sensor Data**

**Avinash Desai**  
**Student Number: 20032381**

**Applied Research Project**

**Submitted in partial fulfilment of the requirements for the degree of MSc.  
Data Analytics**

**at Dublin Business School**



**Supervisor: Sajal Kaur Minhas**

**Module: B9DA113 APPLIED RESEARCH PROJECT (B9DA113\_2425\_TMD3)**

**August 2025**

## Abstract

Predictive maintenance (PdM) is transforming industrial operations by enabling early detection of equipment faults through the analysis of sensor data such as temperature, pressure, and vibration. This approach reduces downtime, extends equipment lifespan, and lowers maintenance costs compared to traditional reactive or scheduled maintenance methods. With the growth of Industry 4.0, machine learning (ML) has become an essential tool in building accurate PdM models.

This study applies multiple ML algorithms—including Logistic Regression, Random Forest, XGBoost, Support Vector Machine (SVM), and K-Nearest Neighbors (KNN)—to predict machine failures using the AI4I 2020 predictive maintenance dataset. The methodology covers data cleaning, preprocessing, feature engineering, class imbalance handling with SMOTE, model training, evaluation, and hyperparameter tuning.

Initial results show that ensemble and boosting models outperform simpler baselines, with potential for further gains through supervised methods. This work provides a reproducible pipeline for industrial fault prediction and lays the foundation for integrating advanced AI techniques in predictive maintenance systems.

## LIST OF FIGURES

|                                                               |    |
|---------------------------------------------------------------|----|
| Figure 1: Project workflow for predictive maintenance .....   | 12 |
| Figure 2: Importing dataset.....                              | 28 |
| Figure 3:Printing Dataset Shape.....                          | 29 |
| Figure 4: Dataset Info .....                                  | 29 |
| Figure 5: Detection of Missing Values.....                    | 30 |
| Figure 6: Identification of Duplicate Entries .....           | 31 |
| Figure 7: Preliminary Analysis of the Target Variable .....   | 32 |
| Figure 8: Establishing a Consistent Visual Style .....        | 33 |
| Figure 9:Code for Target Variable Distribution .....          | 34 |
| Figure 10: Target Variable Distribution .....                 | 34 |
| Figure 11:Code for Feature Correlation Heatmap .....          | 35 |
| Figure 12: Feature Correlation Heatmap .....                  | 36 |
| Figure 13: Code for Failure Rate by Machine Type .....        | 37 |
| Figure 14: Average Machine Failure Rate by Machine Type ..... | 37 |
| Figure 15: Code for Distribution of Numerical Features.....   | 38 |
| Figure 16: Distribution of Air Temperature(K) .....           | 38 |
| Figure 17: Distribution of Process Temperature(K).....        | 39 |
| Figure 18:: Distribution of Rotational Speed(rpm) .....       | 39 |
| Figure 19: Distribution of Torque(Nm).....                    | 40 |

|                                                                             |    |
|-----------------------------------------------------------------------------|----|
| Figure 20: Distribution of Tool Wear(min) .....                             | 40 |
| Figure 21: Code for Outlier Detection via Boxplot .....                     | 41 |
| Figure 22: Boxplot of Process Temperature(K) .....                          | 41 |
| Figure 23: Boxplot of Rotational Speed (rpm) .....                          | 42 |
| Figure 24: Boxplot of Torque(Nm).....                                       | 42 |
| Figure 25: Code to Encode Categorical Features .....                        | 45 |
| Figure 26:Code for scalar and transform .....                               | 46 |
| Figure 27:Code for SMOTE .....                                              | 47 |
| Figure 28: Splitting Data into Training and Testing Sets .....              | 48 |
| Figure 29: Confusion Matrix (Logistic Regression).....                      | 60 |
| Figure 30: ROC Curve and Precision–Recall Curve (Logistic Regression) ..... | 61 |
| Figure 31: Coefficient Importance Plot (Logistic Regression) .....          | 61 |
| Figure 32: Confusion Matrix (Random Forest).....                            | 62 |
| Figure 33: ROC Curve and Precision–Recall Curve (Random Forest) .....       | 63 |
| Figure 34: Feature Importance Plot (Random Forest) .....                    | 63 |
| Figure 35: Confusion Matrix (XGBoost).....                                  | 64 |
| Figure 36: ROC Curve and Precision–Recall Curve (XGBoost) .....             | 65 |
| Figure 37: Feature Importance Plot (XGBoost) .....                          | 65 |
| Figure 38: Confusion Matrix (SVM).....                                      | 66 |
| Figure 39: ROC Curve and Precision–Recall Curve (SVM) .....                 | 67 |
| Figure 40: Confusion Matrix (KNN) .....                                     | 68 |

|                                                             |    |
|-------------------------------------------------------------|----|
| Figure 41: ROC Curve and Precision–Recall Curve (KNN) ..... | 68 |
| Figure 42: Failure Rate by Cluster (K-Means).....           | 69 |
| Figure 43: K-Means Clustering (PCA Projection, k=2).....    | 70 |

## LIST OF TABLES

|                                                                       |    |
|-----------------------------------------------------------------------|----|
| Table 1: First 10 rows of the dataset.....                            | 16 |
| Table 2: Summary of model performance across evaluation metrics ..... | 59 |

## Contents

|                                                    |    |
|----------------------------------------------------|----|
| Abstract .....                                     | 2  |
| LIST OF FIGURES .....                              | 3  |
| LIST OF TABLES .....                               | 6  |
| Chapter 1: Introduction.....                       | 11 |
| 1.1.    Project Overview .....                     | 11 |
| 1.2.    Research Questions.....                    | 13 |
| 1.3.    Problem Statement .....                    | 13 |
| 1.4.    Dataset Overview .....                     | 14 |
| 1.5.    Project Goals and Objectives .....         | 16 |
| 1.6.    Scope and Limitations .....                | 17 |
| 1.7.    Significance of the Study .....            | 18 |
| 1.8.    Environment Setup .....                    | 18 |
| 1.9.    Approach Summary .....                     | 21 |
| 1.10.    Expected Outcomes .....                   | 21 |
| Chapter 2: Literature Review .....                 | 23 |
| Chapter 3: Data Loading & Initial Exploration..... | 28 |
| 3.1. Dataset Importation .....                     | 28 |
| 3.2 Overview of Dataset Structure.....             | 29 |
| 3.3 Examination of Data Types.....                 | 29 |
| 3.4 Detection of Missing Values.....               | 30 |

|                                                           |    |
|-----------------------------------------------------------|----|
| 3.5 Identification of Duplicate Entries .....             | 30 |
| 3.6 Descriptive Statistics Summary .....                  | 31 |
| 3.7 Preliminary Analysis of the Target Variable .....     | 32 |
| 3.8 Summary of Initial Insights.....                      | 32 |
| Chapter 4: Exploratory Data Analysis (EDA).....           | 33 |
| 4.1. Establishing a Consistent Visual Style .....         | 33 |
| 4.2. Target Variable Distribution .....                   | 34 |
| 4.3. Feature Correlation Analysis .....                   | 35 |
| 4.4. Failure Rate by Machine Type .....                   | 36 |
| 4.5. Distribution of Numerical Features.....              | 38 |
| 4.6. Outlier Detection via Boxplots .....                 | 41 |
| 4.7. Summary of EDA Insights .....                        | 42 |
| Chapter 5: Data Preprocessing & Feature Engineering ..... | 44 |
| 5.1. Handling Duplicate Records .....                     | 44 |
| 5.2. Managing Missing Data .....                          | 44 |
| 5.3. Encoding Categorical Features .....                  | 45 |
| 5.4. Feature Scaling .....                                | 46 |
| 5.5. Addressing Class Imbalance with SMOTE .....          | 46 |
| 5.6. Splitting Data into Training and Testing Sets .....  | 47 |
| 5.7. Summary of Data Preprocessing.....                   | 48 |
| Chapter 6: Model Building & Evaluation.....               | 50 |



|                                                                     |    |
|---------------------------------------------------------------------|----|
| 6.1. Overview and goals .....                                       | 50 |
| 6.2. Models selected and why .....                                  | 50 |
| 6.3. Reproducible pipelines and data handling.....                  | 52 |
| 6.4. Baseline training.....                                         | 52 |
| 6.5. Hyperparameter tuning: GridSearchCV explanation .....          | 53 |
| 6.6. Example grids and rationale.....                               | 54 |
| 6.7. Evaluation metrics and interpretation .....                    | 56 |
| 6.8 Visualizations for evaluation .....                             | 57 |
| 6.9 K-means clustering diagnostic .....                             | 57 |
| 6.10. Model selection criteria .....                                | 58 |
| Chapter 7: Results & Interpretation.....                            | 59 |
| 7.1. Logistic Regression .....                                      | 60 |
| 7.2. Random Forest .....                                            | 62 |
| 7.3. XGBoost.....                                                   | 63 |
| 7.4. Support Vector Machine (SVM) .....                             | 65 |
| 7.5. K-Nearest Neighbors (KNN).....                                 | 67 |
| 7.6. K-Means clustering.....                                        | 69 |
| 7.7. Comparative Analysis .....                                     | 70 |
| Chapter 8: Conclusion & Future Work .....                           | 72 |
| 8.1. Conclusion.....                                                | 72 |
| 8.2. Deployment recommendations (designed to maximise recall) ..... | 73 |

|                                                                                    |    |
|------------------------------------------------------------------------------------|----|
| 8.3. Limitations (relevant to recall-focused deployment).....                      | 75 |
| 8.4. Error analysis summary & practical adjustments (reduce false negatives) ..... | 76 |
| 8.5. Future work (actions to raise recall and operationalise safely) .....         | 77 |
| 8.6. Final remark .....                                                            | 78 |
| References.....                                                                    | 79 |

# Chapter 1: Introduction

## 1.1. Project Overview

Predictive maintenance is a growing area of industrial analytics that focuses on forecasting equipment failures before they occur, enabling organizations to take proactive measures and avoid costly downtime. This project addresses a machine failure classification problem, where the goal is to predict whether a machine will fail based on historical operational data.

The dataset used in this project contains records of various sensor readings, operational settings, and maintenance history of machines. Each row in the dataset represents a single instance of machine operation, with associated features such as air temperature, process temperature, rotational speed, torque, tool wear, and more. The target variable is Machine failure, which indicates whether the machine failed (1) or operated normally (0) during the recorded period.

By building and evaluating several machine learning models, this project aims to:

- Accurately classify machine failures.
- Identify the most important predictors of failure.
- Evaluate the impact of imbalanced data on performance.
- Demonstrate how synthetic oversampling techniques (SMOTE) can improve model outcomes.
- Perform hyperparameter tuning to achieve optimal model performance.

The flowchart outlines the end-to-end process: data collection and cleaning, exploratory data analysis and feature engineering, preprocessing (encoding, scaling and SMOTE), modelling

(baseline models, ensemble/boosting, and deep learning), hyperparameter tuning, evaluation (confusion matrix, ROC/PR curves), and interpretation.

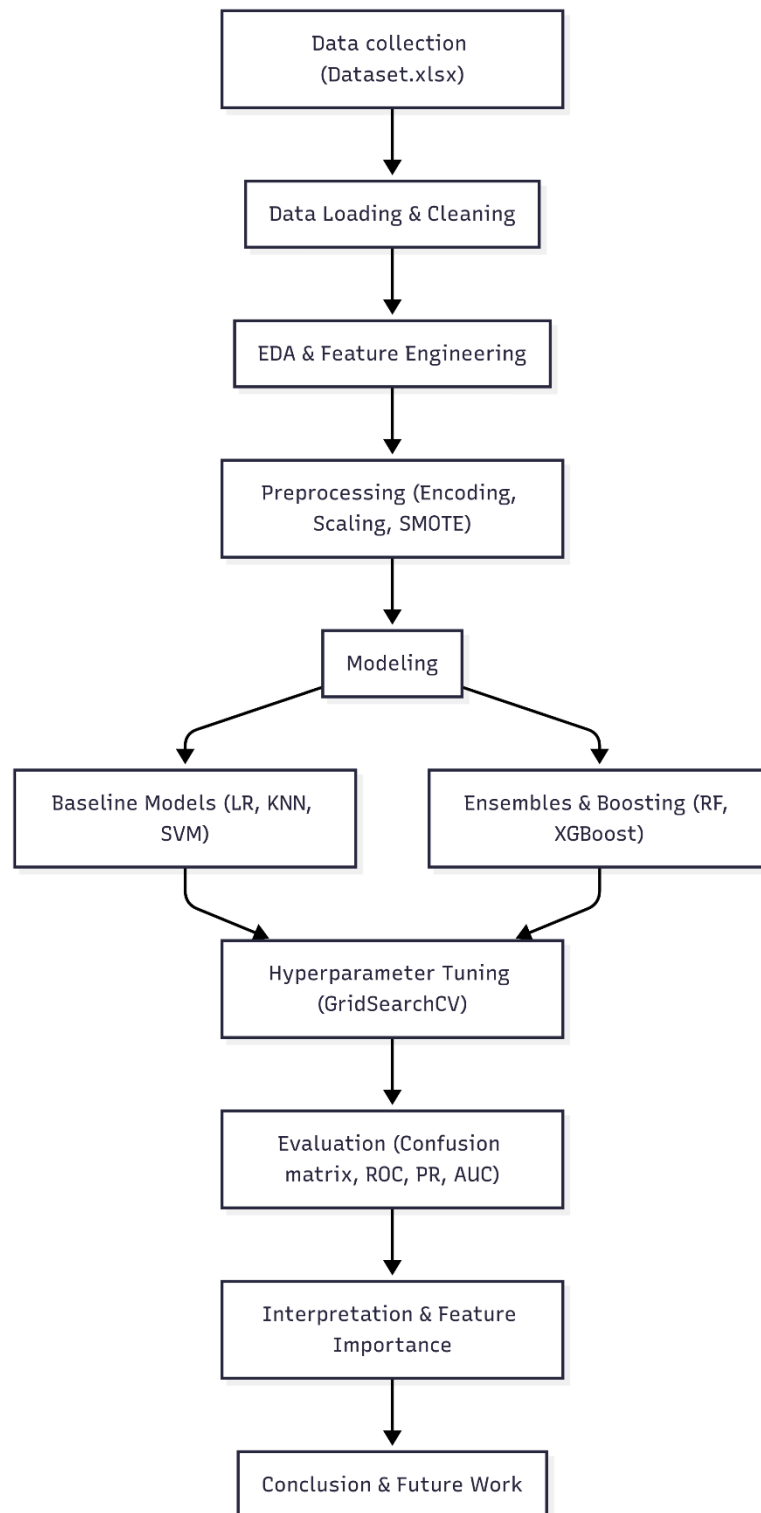


Figure 1: Project workflow for predictive maintenance

## **1.2. Research Questions**

Early Failure Prediction using Sensor Data: To what extent can machine learning models trained on multivariate sensor data—including operational settings, vibration levels, rotational speed, torque, and temperature readings—accurately predict mechanical failures before they occur? This question seeks to evaluate the feasibility and accuracy of using historical and real-time machine telemetry to provide early warnings that allow for proactive intervention, thereby minimizing unplanned downtime and avoiding costly equipment damage.

Enhancing Predictive Maintenance Strategies with Machine Learning: How can machine learning algorithms be leveraged to improve the efficiency, reliability, and cost-effectiveness of predictive maintenance in heavy machinery, particularly by utilizing patterns extracted from vibration and temperature sensor data? This involves assessing the comparative performance of various supervised learning techniques—such as Logistic Regression, Random Forest, XGBoost, SVM, and KNN—in detecting subtle anomalies and degradation trends.

## **1.3. Problem Statement**

Unplanned machine downtime can have significant operational and financial consequences in manufacturing, mining, energy production, and other heavy industries. Traditional maintenance approaches often follow fixed schedules (preventive maintenance) or react to failures after they occur (reactive maintenance). Both strategies have limitations: Preventive maintenance can lead to unnecessary maintenance activities, increasing costs without proportional benefits. Reactive maintenance results in production delays, safety hazards, and higher repair costs.

The problem addressed in this project can be formulated as: Given operational data of machines, predict the likelihood of machine failure to enable proactive maintenance decisions. This is essentially a binary classification problem, where the model outputs either: 0 = No Failure; 1 = Failure.

#### 1.4. Dataset Overview

The dataset (provided in Excel format, Dataset.xlsx) contains several attributes describing machine operation. The dataset contains 10,000 observations and 14 columns where each row represents a single machine record with sensor readings, operational parameters and failure indicators. Identifiers UID and Product ID give row- and product-level identity (the latter encodes the variant letter L / M / H). The core predictive information comprises categorical machine variant (Type), continuous sensor measurements (air/process temperature, rotational speed, torque, tool wear) and five binary failure-mode flags; a final binary Machine failure label is set to 1 if any mode is triggered. Temperatures were simulated with small random-walk noise (air  $\approx 300\text{ K} \pm 2\text{ K}$ ; process  $\approx \text{air} + 10\text{ K} \pm 1\text{ K}$ ), torque is roughly normally distributed around 40 Nm ( $\sigma \approx 10\text{ Nm}$ ), rotational speed is derived from a nominal power signal with added Gaussian noise, and tool wear accumulates with small variant-dependent offsets (H/M/L  $\approx +5/+3/+2$  minutes).

Key dataset details and practical notes

- **Size & structure**
  - 10,000 rows  $\times$  14 columns (including UID, Product ID, sensor columns, five failure flags, and Machine failure target).
  - Product ID encodes variant frequencies: **L = 50%, M = 30%, H = 20%**.

- **Main features**
  - Type (categorical): machine variant (L / M / H).
  - Air temperature [K], Process temperature [K] (continuous).
  - Rotational speed [rpm], Torque [Nm] (continuous).
  - Tool wear [min] (continuous; variant offsets applied).
  - Five binary failure-mode flags: **TWF, HDF, PWF, OSF, RNF**.
- **Failure-mode logic (how Machine failure is derived)**
  - **TWF (Tool Wear Failure)**: tool reaches a randomly chosen replacement/failure time in 200–240 min ( $\approx 120$  events; among these 69 replacements and 51 failures were randomly assigned).
  - **HDF (Heat Dissipation Failure)**: triggered when  $(\text{process\_temp} - \text{air\_temp}) < 8.6 \text{ K}$  **and** rotational speed  $< 1380 \text{ rpm}$  ( $\approx 115$  events).
  - **PWF (Power Failure)**: when torque  $\times$  rotational\_speed (converted to rad/s) yields instantaneous power  $< 3500 \text{ W}$  or  $> 9000 \text{ W}$  ( $\approx 95$  events).
  - **OSF (Overstrain Failure)**: triggered when torque  $\times$  tool\_wear exceeds variant-specific thresholds (L:11,000; M:12,000; H:13,000 min·Nm) ( $\approx 98$  events).
  - **RNF (Random Failure)**: random failures at  $\sim 0.1\%$  probability ( $\approx 5$  events).
  - Machine failure = logical OR of the five flags (so the target indicates *a failure occurred* but not which mode caused it).
- **Counts & class balance**
  - Per-mode event counts are low relative to 10,000 rows (tens–low hundreds per mode), producing a **pronounced class imbalance**: failure events are rare compared to normal operation. Note that modes can overlap, so the total number of rows with Machine failure=1 is not the simple sum of per-mode counts.

Table 1: First 10 rows of the dataset

| UDI | Product ID | Type | Air temperature [K] | Process temperature [K] | Rotational speed [rpm] | Torque [Nm] | Tool wear [min] | Machine failure |
|-----|------------|------|---------------------|-------------------------|------------------------|-------------|-----------------|-----------------|
| 1   | M14860     | M    | 298.1               | 309                     | 1551                   | 42.8        | 0               | 0               |
| 2   | L47181     | L    | 298.2               | 309                     | 1408                   | 46.3        | 3               | 0               |
| 3   | L47182     | L    | 298.1               | 309                     | 1498                   | 49.4        | 5               | 0               |
| 4   | L47183     | L    | 298.2               | 309                     | 1433                   | 39.5        | 7               | 0               |
| 5   | L47184     | L    | 298.2               | 309                     | 1408                   | 40          | 9               | 0               |
| 6   | M14865     | M    | 298.1               | 309                     | 1425                   | 41.9        | 11              | 0               |
| 7   | L47186     | L    | 298.1               | 309                     | 1558                   | 42.4        | 14              | 0               |
| 8   | L47187     | L    | 298.1               | 309                     | 1527                   | 40.2        | 16              | 0               |
| 9   | M14868     | M    | 298.3               | 309                     | 1667                   | 28.6        | 18              | 0               |
| 10  | M14869     | M    | 298.5               | 309                     | 1741                   | 28          | 21              | 0               |

A critical aspect of this dataset is class imbalance — the majority of records belong to the "No Failure" class, with only a small fraction representing failures. This imbalance poses challenges for machine learning models, as they may achieve high accuracy by simply predicting the majority class but fail to identify actual failures.

### 1.5. Project Goals and Objectives

The objectives of this project are as follows:

#### 1. Data Exploration & Understanding

- Examine the dataset structure, types of variables, and basic statistics.
- Identify missing values and duplicates.
- Visualize distributions of features and the target variable.

#### 2. Data Preprocessing

- Remove irrelevant columns (e.g., IDs).
- Encode categorical features.
- Scale numerical features to improve model performance.
- Address class imbalance through oversampling (SMOTE).



### 3. Model Training & Evaluation

- Train multiple machine learning models: Logistic Regression, Random Forest, XGBoost, Support Vector Machines (SVM), and K-Nearest Neighbors (KNN).
- Evaluate models using classification metrics (precision, recall, F1-score, ROC AUC).
- Visualize confusion matrices and ROC curves.

### 4. Hyperparameter Tuning

- Use GridSearchCV to find the best hyperparameters for selected models.
- Compare tuned models against their default configurations.

### 5. Insights & Recommendations

- Identify features most strongly associated with machine failure.
- Provide actionable insights for maintenance teams.
- Highlight limitations and propose future improvements.

## 1.6. Scope and Limitations

#### Scope:

- The project is limited to supervised machine learning classification methods.
- Only the provided dataset is used for model training and evaluation.
- The models are trained and tested on historical data — real-time streaming prediction is not covered.

#### Limitations:

- Class imbalance may still affect model generalization, even after applying SMOTE.
- The dataset may not capture all real-world conditions leading to machine failure.
- The models may be sensitive to hyperparameter settings and data preprocessing choices.

- External factors such as environmental conditions, operator behaviour, or manufacturing defects are not included in the dataset.

### 1.7. Significance of the Study

The implementation of predictive maintenance through machine learning offers substantial benefits:

1. **Cost Savings:** Preventing unplanned downtime reduces maintenance costs and production losses.
2. **Increased Safety:** Early failure prediction reduces the risk of accidents caused by sudden equipment breakdown.
3. **Operational Efficiency:** Maintenance resources can be allocated optimally, focusing on high-risk machinery.
4. **Scalability:** Once trained, the models can be integrated into IoT-enabled monitoring systems to continuously assess machine health.

This project provides a blueprint for developing predictive maintenance solutions using supervised machine learning, applicable across multiple industries.

### 1.8. Environment Setup

Establishing a consistent and reproducible computing environment is a crucial first step in any machine learning project. It ensures that the analysis and results can be reliably duplicated by others, regardless of the hardware or platform used. This section outlines the key considerations and choices made to prepare the environment for this project.

#### 1. Selection of Development Environment

Multiple options exist for running Python-based machine learning workflows, each with its own advantages:

- **Cloud-based Platforms (e.g., Google Colab):** These platforms provide the convenience of running code in a browser without any local setup. They often offer free access to hardware accelerators such as GPUs or TPUs, which are beneficial for training complex models. Cloud environments also facilitate easy sharing and collaboration through notebook links.
- **Local Jupyter Notebooks:** Running Jupyter on a personal computer offers more direct control over files and dependencies. It is suitable for users who prefer working offline or need to access local data. However, it requires manual installation and management of Python and related libraries.
- **Integrated Development Environments (IDEs) like VS Code or PyCharm:** These provide robust tools for larger projects, including advanced debugging, project organization, and seamless integration with version control systems. IDEs are well-suited for production-level code development beyond interactive notebooks.

For this project, Google Colab was chosen as the primary environment due to its accessibility, built-in GPU support, and ease of sharing results. The workflows developed are compatible with local Jupyter notebooks, enabling flexibility for future use.

## 2. Python Version and Library Requirements

The project was developed using Python version 3.9 or higher. This ensures compatibility with modern machine learning libraries and takes advantage of recent performance improvements and features.

A selection of widely used libraries was employed, covering data manipulation, visualization, machine learning algorithms, and handling dataset imbalances. These include tools for statistical analysis, plotting, and implementation of various classification algorithms. Ensuring the availability and compatibility of these libraries is essential for a smooth development process.

### 3. Dataset Management

The dataset used in this study was provided in Excel format. Appropriate tools were utilized to read and process this file format efficiently. In cloud environments, datasets can be uploaded directly via the interface or mounted from external storage services. In local setups, direct file system access facilitates dataset management.

### 4. Project Organization and Reproducibility

Maintaining an organized project structure is key to reproducibility and collaboration. This involves clearly dividing the analysis into logical stages such as environment setup, data loading, preprocessing, modeling, evaluation, and interpretation.

Version control systems, such as Git, are recommended for tracking changes over time and coordinating among multiple collaborators. Additionally, recording exact library versions and dependencies ensures that others can recreate the environment accurately.

### 5. Visualization Consistency

To improve clarity and readability, consistent styles and themes for plots and charts were adopted throughout the project. This practice aids in producing visually coherent reports and presentations.

## 1.9. Approach Summary

The workflow followed in this project can be summarized in five major steps:

### 1. Data Loading and Exploration

- Load the dataset using Pandas.
- Perform an initial inspection (.head(), .info(), .describe()).
- Explore the target variable distribution to assess imbalance.

### 2. Preprocessing and Feature Engineering

- Drop irrelevant columns (UDI, Product ID).
- Encode categorical variables using Label Encoding.
- Apply Standard Scaler to normalize numerical features.
- Split into training and testing sets (stratified sampling to maintain class ratio).
- Apply SMOTE on the training set to handle imbalance.

### 3. Model Training

- Train baseline models without tuning.
- Compare models based on classification metrics.
- Use ROC curves for visual comparison.

### 4. Hyperparameter Optimization

- Apply GridSearchCV to improve selected models.
- Select the best-performing hyperparameters.

### 5. Evaluation and Insights

- Compare tuned models vs. baseline models.
- Identify most important features for machine failure prediction.
- Summarize findings and limitations.

## 1.10. Expected Outcomes

By the end of this project, we expect to achieve:

- An understanding of the dataset structure and potential predictive variables.

- A set of well-performing machine learning models for predicting machine failure.
- Evidence that handling class imbalance improves recall for the minority class.
- Insights into which machine characteristics are most critical for failure prediction.
- A documented process that can be adapted to similar predictive maintenance use cases.

## Chapter 2: Literature Review

Transfer Learning in Predictive Maintenance, Azari et al. (2023) explore the application of transfer learning for predictive maintenance in Industry 4.0 environments. Transfer learning, which involves leveraging pre-trained models to predict faults in new machinery, has shown promise in scenarios where labeled data is scarce. The study highlights how transfer learning can improve fault detection by applying knowledge gained from similar systems to predict failures in different machinery, thus enhancing the model's generalization and accuracy. This approach is particularly relevant for PdM systems in heavy machinery, where acquiring labeled data for every machine type can be resource-intensive.

IoT-Enabled Machine Learning, Madasamy et al. (2023) discuss the role of IoT in enabling machine learning for predictive maintenance in Industry 4.0. IoT devices such as vibration and temperature sensors generate large volumes of real-time data, which can be processed using machine learning algorithms to predict potential equipment failures. The study emphasizes that the integration of these sensors with advanced machine learning techniques, such as deep learning, can lead to more accurate and timely predictions of maintenance needs. IoT-enabled predictive maintenance offers a scalable and efficient solution for monitoring the health of heavy machinery.

Vibration Analysis for Predictive Maintenance, Pavithra and Ramachandran (2021) provide an overview of how vibration analysis has been used for predictive maintenance in industrial machinery. Vibration data is often indicative of mechanical faults, such as imbalances, misalignments, or bearing wear. Machine learning models, particularly those using time-series analysis, can analyze vibration signals to detect anomalies that may signal impending failures.

This approach has been widely adopted in heavy machinery industries due to the direct relationship between vibrations and mechanical health.

Uncertainty-Aware Deep Learning, Shao et al. (2023) introduce uncertainty-aware deep learning as a promising tool for fault diagnosis in industrial applications. Their study focuses on the incorporation of uncertainty quantification into deep learning models to enhance the reliability of predictive maintenance systems. Uncertainty-aware models can better handle noisy sensor data, such as that obtained from temperature and vibration sensors, improving the robustness and trustworthiness of fault detection systems. This is crucial in real-world industrial settings where sensor data can be subject to various sources of uncertainty.

Hybrid Deep Learning Models for Fault Diagnosis Zabin et al. (2022) propose a hybrid deep transfer learning architecture that combines Hilbert transform and deep convolutional neural networks (DCNN) with long short-term memory (LSTM) networks for industrial fault diagnosis. This approach leverages both time-domain and frequency-domain features extracted from vibration signals, improving fault classification accuracy. By combining these models with temperature data, which provides complementary insights into thermal anomalies, such hybrid systems can offer a more comprehensive analysis of machine health, thus enabling more accurate predictive maintenance solutions.

Sridhar and Sanagavarapu (2012) investigated the challenge of class imbalance in predictive maintenance datasets, where machine failure cases are rare compared to normal operations. They demonstrated that applying SMOTE-based oversampling can significantly improve the performance of machine learning models in classifying failures. Their work emphasizes the importance of preprocessing techniques in handling skewed datasets, which directly aligns



with the imbalance issue in this study's dataset. This provides strong justification for incorporating SMOTE in the proposed workflow to enhance recall for the minority failure class.

Syed et al. (2020) proposed a novel linear classification method tailored to address imbalance in failure-prone air pressure systems. Their experiments revealed that traditional classifiers often achieve misleadingly high accuracy by favoring the majority class, while their approach improved detection of rare failures. The study highlights that imbalance-aware classifiers can provide superior fault prediction performance compared to standard models. This insight supports the exploration of both data-level (SMOTE) and algorithm-level (cost-sensitive learning) solutions in the current project.

Hafeez et al. (2021) developed a sequential multivariate fault prediction model for vehicular predictive maintenance using machine learning. Unlike static models, their approach captured temporal dependencies in sensor data to anticipate failures more accurately. The study demonstrates the value of time-series modeling and feature dependencies in PdM tasks. While the present project uses a static dataset, their findings suggest potential future extensions into sequential modeling for real-time failure prediction.

Susto et al. (2015) presented a multiple classifier strategy for predictive maintenance in industrial systems, integrating various models to capture different health states of machines. Their framework combined classification with failure horizon estimation, enabling maintenance teams to anticipate not just whether a failure would occur, but also when. This work reinforces the importance of using ensemble and hybrid models, which parallels the current study's evaluation of Random Forest, XGBoost, and other ensemble approaches for failure classification.

Lorenti et al. (2023) conducted a comparative study on deep learning models for Remaining Useful Life (RUL) estimation in predictive maintenance. Their work evaluated multiple deep architectures, including recurrent neural networks (RNNs), convolutional networks (CNNs), and hybrid approaches, across different industrial case studies. The results showed that while deep learning models provide superior accuracy in capturing nonlinear degradation patterns, performance is highly dependent on the choice of architecture, feature representation, and hyperparameter tuning. Importantly, the study highlighted challenges such as model interpretability and computational cost, which limit industrial adoption. This paper provides strong evidence for exploring deep learning alongside traditional machine learning classifiers, and it supports the current thesis objective of benchmarking multiple models for predictive maintenance while considering trade-offs between performance and complexity.

Recent research in predictive maintenance (PdM) has explored a variety of approaches to enhance fault detection and Remaining Useful Life (RUL) estimation across industrial systems. Azari et al. (2023) demonstrated the effectiveness of transfer learning in scenarios with limited labeled data, enabling knowledge transfer between similar machines. Madasamy et al. (2023) emphasized the integration of IoT sensor data with machine learning to provide scalable and real-time PdM solutions, while Pavithra and Ramachandran (2021) highlighted the importance of vibration analysis and time-series modeling in detecting mechanical faults. To improve robustness, Shao et al. (2023) proposed uncertainty-aware deep learning models capable of handling noisy industrial sensor data. Similarly, Zabin et al. (2022) introduced hybrid deep learning architectures that combine CNN, LSTM, and frequency-domain features for superior fault classification accuracy.

Parallel to these advances, other studies have addressed challenges in data characteristics. Sridhar and Sanagavarapu (2012) and Syed et al. (2020) focused on the class imbalance problem, showing that SMOTE-based oversampling and imbalance-aware classifiers can significantly improve rare fault detection. Hafeez et al. (2021) proposed a sequential multivariate model to capture temporal dependencies in vehicular maintenance data, stressing the importance of time-series approaches. In addition, Susto et al. (2015) demonstrated the potential of ensemble classifier frameworks to predict both fault occurrence and timing, while Lorenti et al. (2023) evaluated deep learning architectures for RUL estimation, underscoring trade-offs between accuracy, interpretability, and computational cost.

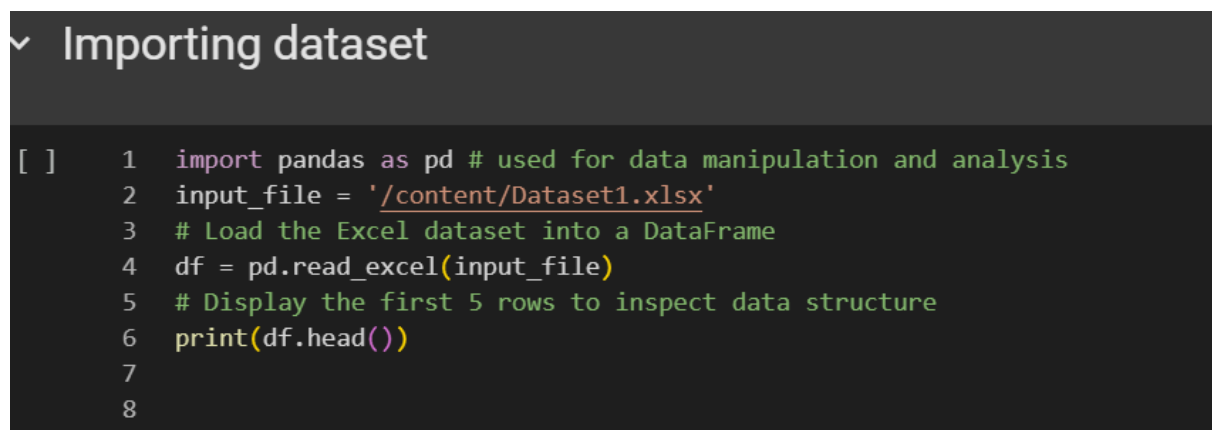
Together, these studies illustrate the evolution of PdM research from traditional imbalance-handling methods to advanced deep and hybrid learning strategies. However, gaps remain in balancing accuracy, interpretability, and computational efficiency, particularly for imbalanced industrial datasets—an area directly addressed in this thesis through a comparative evaluation of machine learning models combined with oversampling techniques.

## Chapter 3: Data Loading & Initial Exploration

This chapter focuses on importing the dataset, gaining a preliminary understanding of its structure, and performing initial quality checks. The purpose is to establish a clear overview of the data before proceeding to any cleaning, transformation, or modelling steps.

### 3.1. Dataset Importation

The dataset, provided in Excel format, is loaded into the working environment using appropriate data handling tools. Depending on the chosen platform (cloud-based or local), the file needs to be accessible and correctly referenced for smooth ingestion. We begin by importing the dataset, which is provided in Excel format. We use pandas to read the Excel file into a DataFrame. This gives us a structured table in Python for analysis. After loading, we inspect the first few rows to verify that the data appears correctly.



```
[ ] 1 import pandas as pd # used for data manipulation and analysis
    2 input_file = '/content/Dataset1.xlsx'
    3 # Load the Excel dataset into a DataFrame
    4 df = pd.read_excel(input_file)
    5 # Display the first 5 rows to inspect data structure
    6 print(df.head())
    7
    8
```

*Figure 2: Importing dataset*

Once imported, a brief look at the first few records offers a snapshot of the dataset's structure, including column names, sample values, and data consistency. This initial inspection helps confirm that the data has been correctly loaded and aligns with expectations.

### 3.2 Overview of Dataset Structure

Understanding the dataset's dimensions—how many rows (observations) and columns (features) it contains—is fundamental. Listing all column names provides insight into the types of variables available for analysis and modelling.

```
1 # Print dataset shape (rows, columns) and list of column names
2 print("Shape:", df.shape)
3 print("Columns:", list(df.columns))
4
```

*Figure 3: Printing Dataset Shape*

This step sets the stage for identifying which features may be relevant, redundant, or in need of special treatment.

### 3.3 Examination of Data Types

Each feature's data type is reviewed to determine whether it is numerical, categorical, or of another format. This classification informs the preprocessing strategy:

- Numerical features may require scaling or normalization.
- Categorical features typically need encoding to be usable in machine learning models.

```
1 # Get data type and non-null count for each column
2 df.info()
3
```

*Figure 4: Dataset Info*

Understanding data types early helps avoid errors and guides subsequent data transformation choices.

### 3.4 Detection of Missing Values

Missing data is a common challenge that can impact model performance. This phase involves quantifying the extent of missingness in each feature, both in absolute terms and as a percentage of total observations.

```
# Calculate missing values count and percentage for each column
missing = df.isnull().sum()
missing_percent = (missing / len(df)) * 100
print(pd.concat([missing, missing_percent], axis=1, keys=['Missing', '%']))
```

*Figure 5: Detection of Missing Values*

Depending on the nature and amount of missing data, several strategies can be employed:

- Removing incomplete records,
- Imputing missing values with statistical measures (mean, median, mode),
- Or applying advanced imputation techniques.

The chosen approach balances data integrity with model accuracy.

### 3.5 Identification of Duplicate Entries

Duplicate records can distort analysis and lead to biased models. Therefore, duplicates are identified and quantified. If duplicates are present, they may be removed to ensure each observation represents unique information.

```

1  # Check for duplicates in the entire DataFrame
2  duplicate_rows = df[df.duplicated()]
3
4  # Display the duplicate rows
5  print(f"Number of duplicate rows: {duplicate_rows.shape[0]}")
6  duplicates = df.duplicated().sum()
7  print(f"Duplicate records: {duplicates}")
8  print(duplicate_rows.head())
9
10 # Remove duplicate rows (if any)
11 df = df.drop_duplicates().reset_index(drop=True)

```

*Figure 6: Identification of Duplicate Entries*

This step helps maintain the quality and validity of the dataset.

### 3.6 Descriptive Statistics Summary

Basic statistical summaries are computed for numerical features to understand their distribution and central tendencies. Key metrics include:

- Count of non-missing values,
- Mean and standard deviation,
- Minimum and maximum values,
- Quartiles (25th, 50th/median, 75th percentiles).

*# Basic descriptive statistics for numeric columns*

```
print(df.describe())
```

These statistics provide a foundation for deeper exploratory analysis and highlight potential anomalies or outliers.

### 3.7 Preliminary Analysis of the Target Variable

A quick examination of the target variable, such as “Machine failure,” includes counting the occurrences of each class and calculating their relative proportions. This step is crucial for detecting class imbalance, which influences the choice of evaluation metrics and resampling techniques later in the modelling process.

```
1 # Display counts and percentages for target classes
2 target_counts = df['Machine failure'].value_counts()
3 target_percent = df['Machine failure'].value_counts(normalize=True) * 100
4 print(pd.concat([target_counts, target_percent], axis=1, keys=['Count', '%']))
5
```

*Figure 7: Preliminary Analysis of the Target Variable*

### 3.8 Summary of Initial Insights

At the conclusion of this initial data exploration, we have established:

- The overall size and shape of the dataset,
- The data type classification for all features,
- The presence and extent of any missing values or duplicates,
- Basic descriptive statistics of numeric columns,
- The distribution and balance of the target variable classes.

This foundational understanding provides confidence to proceed with more detailed exploratory data analysis, where visualizations and correlation studies will uncover deeper patterns and relationships within the data.



## Chapter 4: Exploratory Data Analysis (EDA)

The objective of Exploratory Data Analysis is to gain a deeper understanding of the dataset by visually and statistically examining the data. This process helps uncover patterns, relationships, and potential issues, which are essential for guiding effective data preprocessing and model development.

In this chapter, we will:

- Analyse the distribution of the target variable,
- Investigate correlations among features,
- Explore failure rates across different machine types,
- Examine the distribution and characteristics of individual features.

### 4.1. Establishing a Consistent Visual Style

To ensure clarity and coherence throughout our visual analysis, a consistent graphical style is applied to all plots. This uniformity enhances readability and makes the interpretation of visualizations more straightforward.

```
1 import matplotlib.pyplot as plt
2 import seaborn as sns
3 # Set a consistent style for all plots
4 sns.set_style("whitegrid")
5 sns.set(style="whitegrid")
6
```

*Figure 8: Establishing a Consistent Visual Style*

## 4.2. Target Variable Distribution

The first step is to examine the distribution of the target variable, “Machine failure.” Understanding the balance between failure and non-failure instances is critical because imbalanced classes can influence the choice of modelling techniques and evaluation metrics.

```
1 # Target Variable Distribution
2 # This shows how many machines failed vs. how many did not
3 plt.figure(figsize=(6, 4))
4 sns.countplot(x='Machine failure', data=df, palette="Set2")
5 plt.title("Target Variable Distribution (Machine Failure)", fontsize=14)
6 plt.xlabel("Machine Failure (0 = No, 1 = Yes)")
7 plt.ylabel("Count")
8 plt.xticks([0, 1], ["No Failure", "Failure"])
9 plt.tight_layout()
10 plt.show()
11 # This plot helps to understand class imbalance.
12
```

Figure 9: Code for Target Variable Distribution

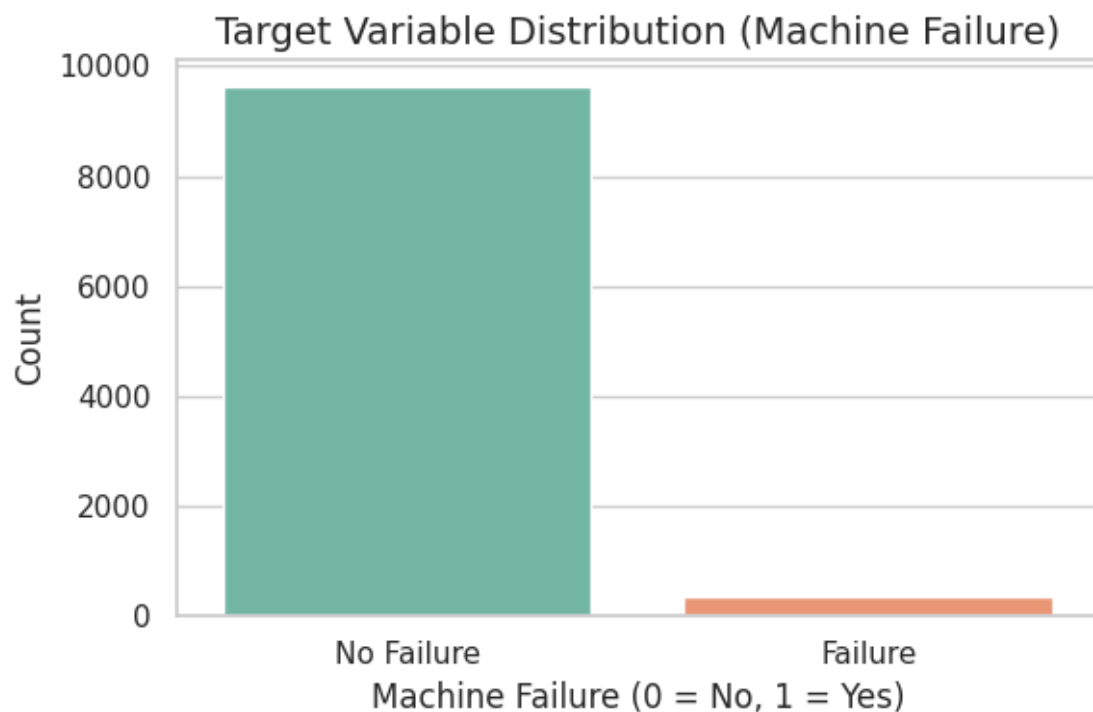


Figure 10: Target Variable Distribution

An imbalanced dataset, where one class significantly outnumbers the other, may require specialized methods such as resampling or adjusting performance measures to ensure the model's robustness and fairness.

### 4.3. Feature Correlation Analysis

Next, correlations between numerical features are explored using a correlation matrix. This matrix quantifies the strength and direction of linear relationships between variables.

Key points to consider include:

- Strong positive or negative correlations between features may suggest redundancy or multicollinearity, which can impact model performance, especially in algorithms sensitive to correlated inputs (e.g., Logistic Regression).
- High correlation between a feature and the target variable indicates potential predictive power.

```
1  # Correlation Heatmap
2  # Visualize pairwise correlation between numerical features
3  plt.figure(figsize=(12, 8))
4
5  # Compute correlation matrix
6  correlation_matrix = df.corr(numeric_only=True)
7
8  # Plot heatmap with annotations
9  sns.heatmap(correlation_matrix,
10             annot=True,
11             fmt=".2f",
12             cmap="coolwarm",
13             square=True,
14             linewidths=0.5,
15             cbar_kws={"shrink": 0.8})
16
17 plt.title("Feature Correlation Heatmap", fontsize=16)
18 plt.tight_layout()
19 plt.show()
```

Figure 11: Code for Feature Correlation Heatmap

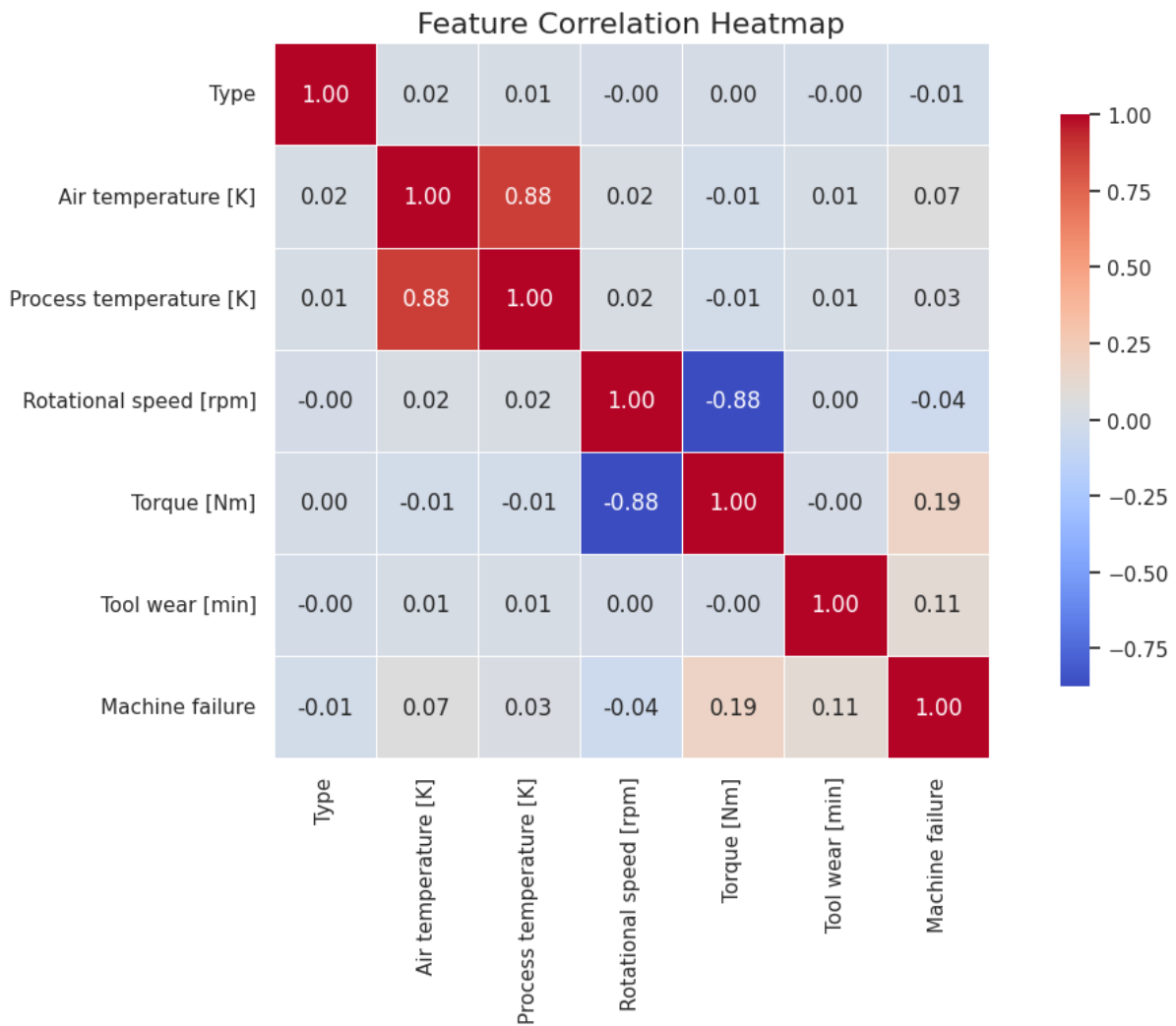


Figure 12: Feature Correlation Heatmap

Identifying these relationships aids in feature selection and engineering decisions.

#### 4.4. Failure Rate by Machine Type

Since machine types are encoded categorically (e.g., L=0, M=1, H=2), we analyse how failure rates differ across these categories. This information can highlight specific machine classes that are more prone to failure.

```

1 # Failure Rate by Machine Type
2 # Visualizes how machine failure rate varies by 'Type' (already label encoded)
3 plt.figure(figsize=(6, 4))
4
5 # Plot average failure rate per machine type
6 sns.barplot(x="Type", y="Machine failure", data=df, palette="pastel", ci=None)
7
8 plt.title("Average Machine Failure Rate by Machine Type", fontsize=14)
9 plt.xlabel("Machine Type (Encoded: L=0, M=1, H=2)")
10 plt.ylabel("Failure Rate")
11 plt.xticks([0, 1, 2], ['L', 'M', 'H']) # Map back to original labels for clarity
12 plt.tight_layout()
13 plt.show()

```

Figure 13: Code for Failure Rate by Machine Type

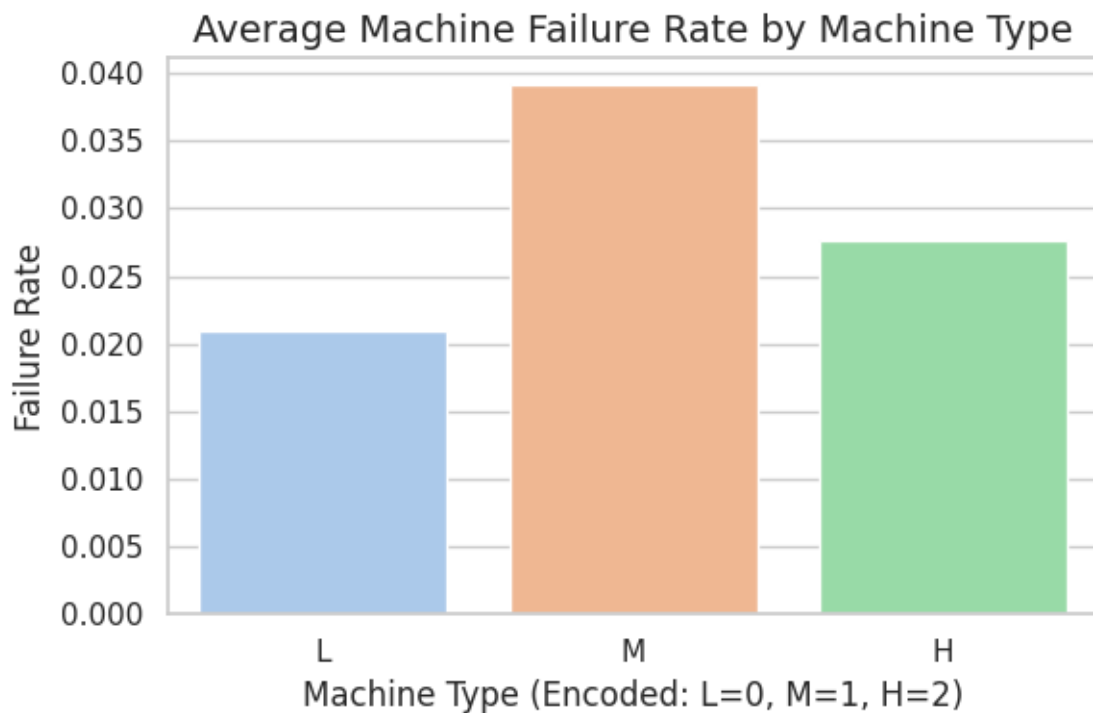


Figure 14: Average Machine Failure Rate by Machine Type

Such insights are valuable for targeting maintenance efforts, optimizing resource allocation, and reducing downtime in practical settings.

## 4.5. Distribution of Numerical Features

Investigating the distribution of each numerical feature helps detect:

- Skewness: Non-normal distributions may require transformation (e.g., logarithmic) to improve model performance.
- Outliers: Extreme values can disproportionately influence some models and may need to be addressed.

```
1 numeric_cols = df.select_dtypes(include=['float64', 'int64']).columns
2 for col in numeric_cols:
3     plt.figure(figsize=(6,4))
4     sns.histplot(df[col], kde=True)
5     plt.title(f"Distribution of {col}")
6     plt.xlabel(col)
7     plt.ylabel("Frequency")
8     plt.show()
9
```

Figure 15: Code for Distribution of Numerical Features

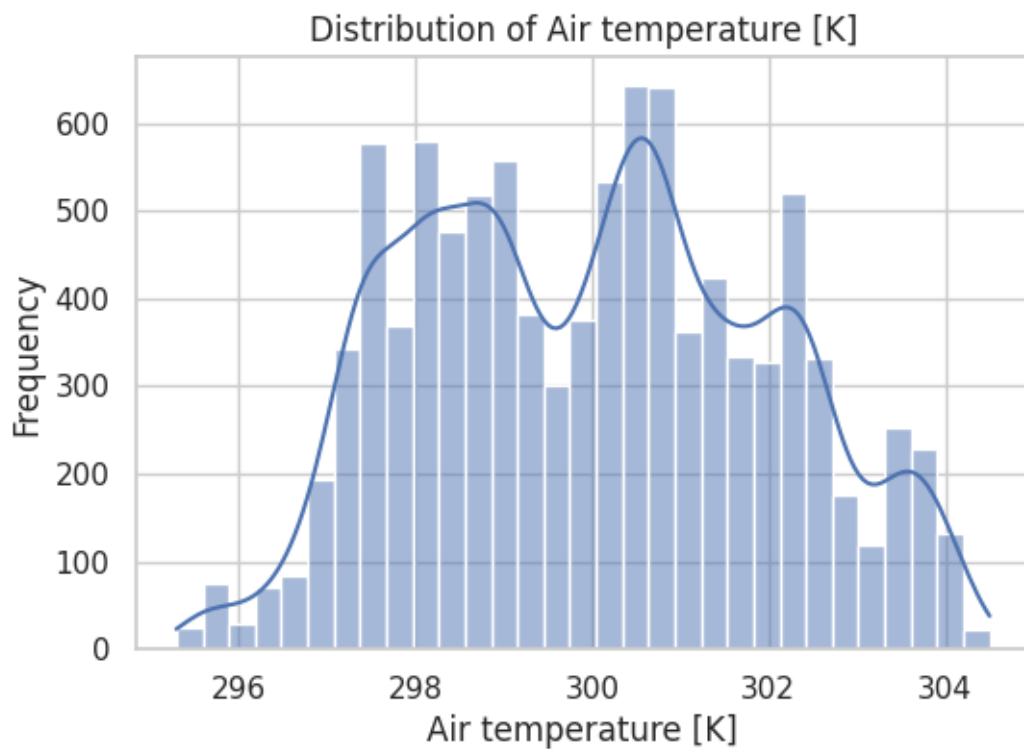


Figure 16: Distribution of Air Temperature(K)

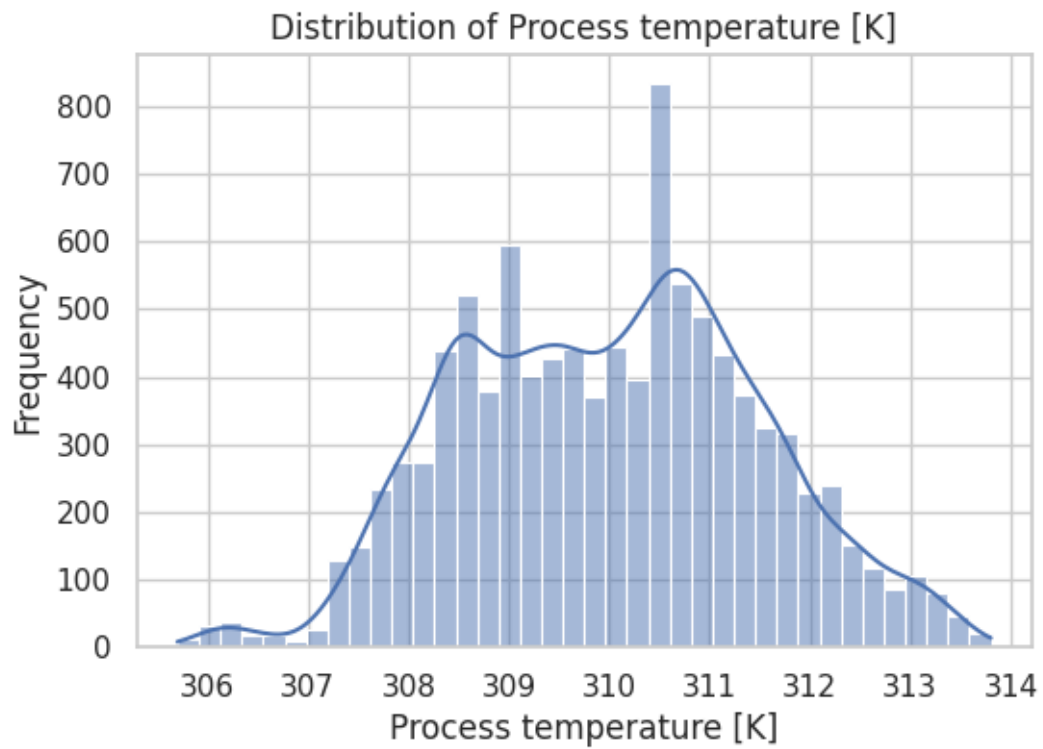


Figure 17: Distribution of Process Temperature(K)

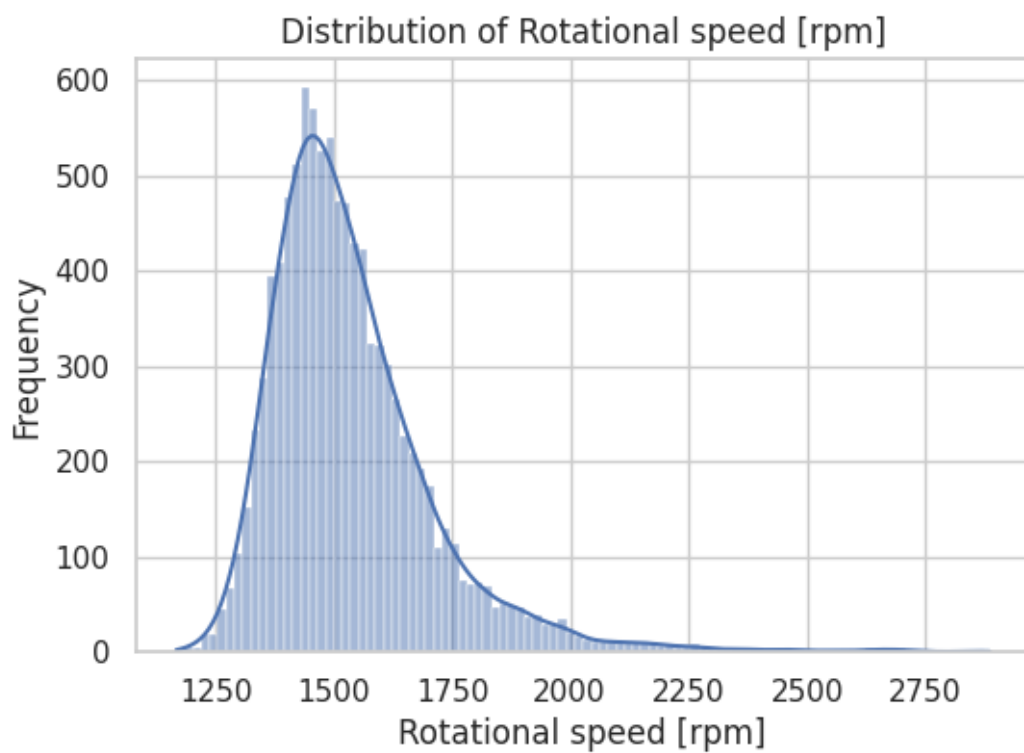


Figure 18:: Distribution of Rotational Speed(rpm)

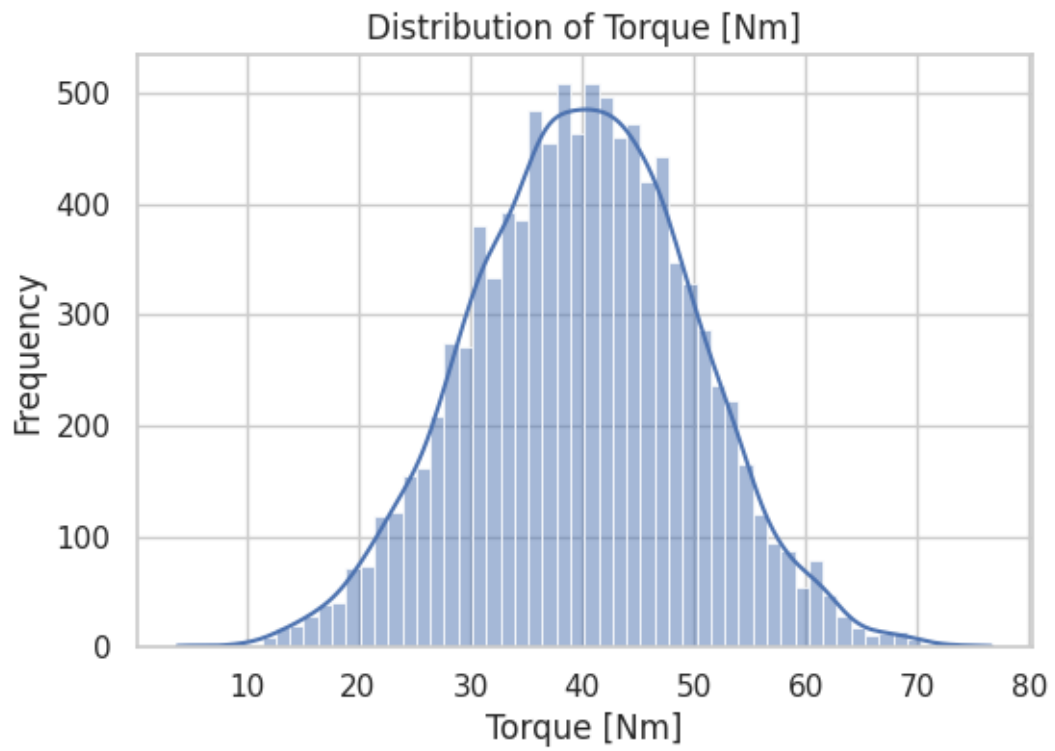


Figure 19: Distribution of Torque(Nm)

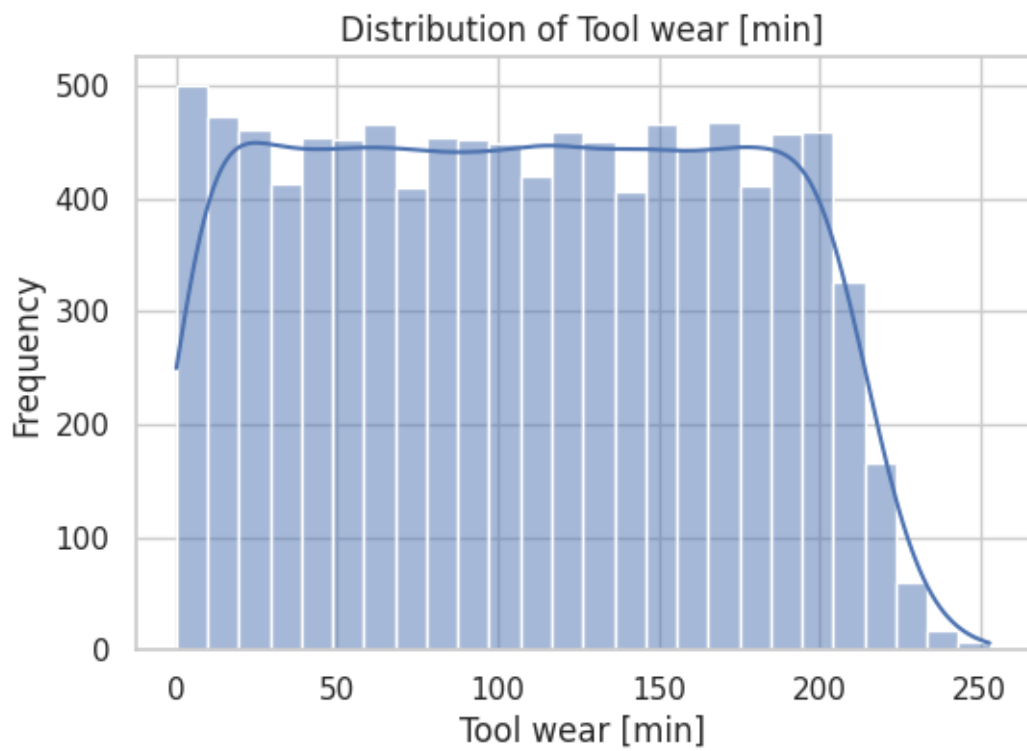


Figure 20: Distribution of Tool Wear(min)



Visualizations such as histograms with density curves provide an intuitive understanding of data shape and variability.

#### 4.6. Outlier Detection via Boxplots

Boxplots offer a concise summary of the spread and outliers within numerical features.

Outliers are data points that fall outside the typical range and can sometimes represent measurement errors, rare events, or important variability.

```
for col in numeric_cols:
    plt.figure(figsize=(6,2))
    sns.boxplot(x=df[col])
    plt.title(f"Boxplot of {col}")
    plt.xlabel(col)
    plt.show()
```

Figure 21: Code for Outlier Detection via Boxplot

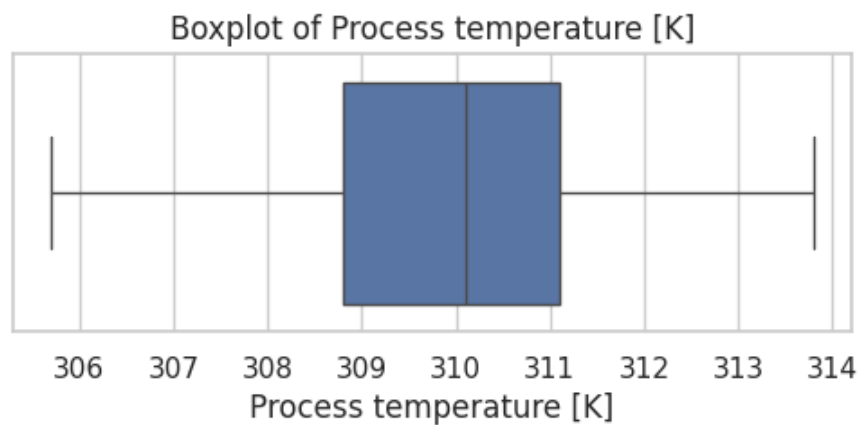


Figure 22: Boxplot of Process Temperature(K)

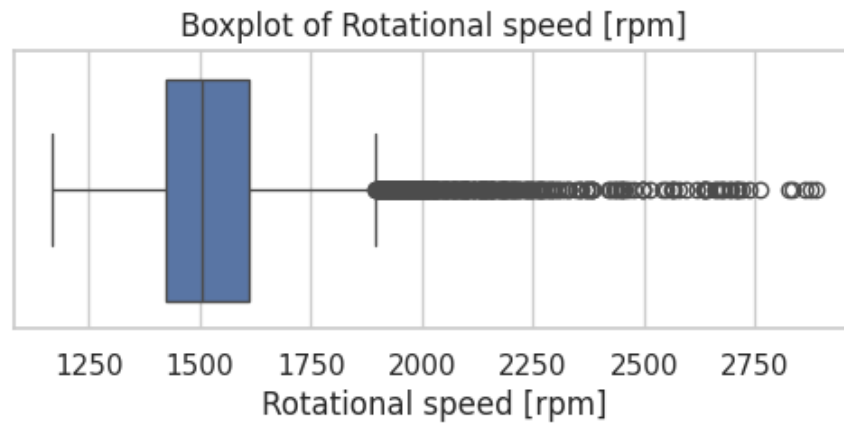


Figure 23: Boxplot of Rotational Speed (rpm)

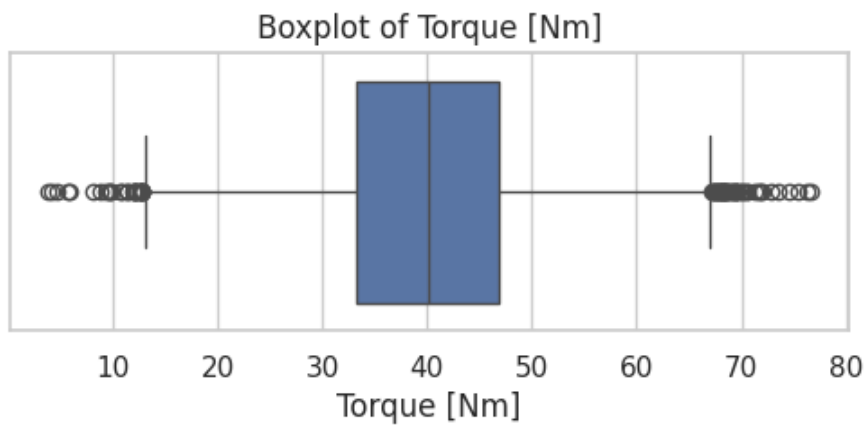


Figure 24: Boxplot of Torque(Nm)

While not all outliers should be removed, their presence must be acknowledged and considered in model design and feature processing.

#### 4.7. Summary of EDA Insights

At the conclusion of this exploratory analysis, we will have established:

- Whether the target variable exhibits class imbalance,
- Which features demonstrate strong inter-correlations or correlation with the target,
- Variations in failure rates across machine types,
- The presence of skewed distributions or outliers in numerical features,

- Candidate features that warrant further attention for predictive modelling.

These findings provide a crucial foundation for the subsequent chapter focused on data preprocessing and feature engineering, ensuring the dataset is optimally prepared for machine learning.

## Chapter 5: Data Preprocessing & Feature Engineering

Before building machine learning models, it is essential to prepare the raw data through cleaning, transformation, and formatting so that models can effectively learn from it. This chapter outlines the key preprocessing steps necessary to ensure high-quality input for predictive modelling. These include:

- Removing duplicate entries,
- Handling missing values,
- Encoding categorical variables into numeric format,
- Scaling numerical features,
- Addressing class imbalance,
- Splitting data into training and testing subsets.

### 5.1. Handling Duplicate Records

Duplicate data points can bias the training process by giving undue influence to repeated observations. Identifying and removing these duplicates helps maintain the integrity and fairness of the dataset, ensuring that each data point contributes uniquely to the learning process.

```
# Remove duplicates and reset index
```

```
df = df.drop_duplicates().reset_index(drop=True)
```

### 5.2. Managing Missing Data

Missing values can arise due to sensor malfunctions, logging errors, or other data collection issues. It is crucial to address these gaps to prevent downstream errors in model training.

Common strategies include:

- Filling missing numerical values with the median, which is less sensitive to outliers than the mean.
- Replacing missing categorical values with the mode (the most frequently occurring category), providing a reasonable default estimate.

These imputation techniques help maintain dataset completeness while minimizing distortion.

### 5.3. Encoding Categorical Features

Machine learning algorithms require all input features to be numeric. Therefore, categorical variables—such as machine types or status labels—must be converted into numeric codes.

```
1 !pip install scikit-learn
2 from sklearn.preprocessing import LabelEncoder
3 label_encoder = LabelEncoder()
4 df['Type'] = label_encoder.fit_transform(df['Type'])
5
6 # Show mapping of labels
7 type_mapping = dict(zip(label_encoder.classes_, label_encoder.transform(label_encoder.classes_)))
8 print("\nLabel encoding mapping for 'Type':", type_mapping)
```

*Figure 25: Code to Encode Categorical Features*

Label encoding assigns a unique integer to each category, preserving the categorical distinctions while enabling models to process them. Importantly, the mappings are stored so that encoded values can be interpreted or reversed after prediction.

## 5.4. Feature Scaling

Many machine learning models, particularly distance-based or gradient-based ones (e.g., Logistic Regression, Support Vector Machines, K-Nearest Neighbors), perform better when input features are on a comparable scale.

```
1  # Initialize the scaler
2  from sklearn.preprocessing import StandardScaler
3  scaler = StandardScaler()
4
5  # Fit the scaler on features (X) and transform them
6  #   This standardizes the data: mean = 0, standard deviation = 1
7  X_scaled = scaler.fit_transform(X)
8
9  # Convert scaled array back to a DataFrame for better readability
10 X_scaled_df = pd.DataFrame(X_scaled, columns=X.columns)
11 print("\nScaled feature sample:")
12 print(X_scaled_df.head())
```

*Figure 26: Code for scalar and transform*

Standardization—scaling features to have zero mean and unit variance—is a common approach that improves convergence speed and model stability. Although tree-based models (e.g., Random Forest, XGBoost) are less sensitive to feature scales, applying uniform scaling ensures consistency when testing multiple algorithms.

## 5.5. Addressing Class Imbalance with SMOTE

When the target variable classes are imbalanced (e.g., significantly fewer failure events than non-failures), models may become biased towards the majority class.

```

1  from imblearn.over_sampling import SMOTE
2
3  # 1. Initialize SMOTE
4  smote = SMOTE(random_state=42)
5
6  # 2. Apply SMOTE only to the training data (NOT the test data)
7  X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
8
9  # 3. Show the new class distribution
10 print("\n Class distribution AFTER applying SMOTE (on training set):")
11 print(pd.Series(y_train_resampled).value_counts())
12
13 # Compare shapes
14 print(f"\nOriginal X_train shape: {X_train.shape}")
15 print(f"Resampled X_train shape: {X_train_resampled.shape}")
16

```

```

Class distribution AFTER applying SMOTE (on training set):
Machine failure
0    6737
1    6737
Name: count, dtype: int64

Original X_train shape: (7000, 6)
Resampled X_train shape: (13474, 6)

```

Figure 27: Code for SMOTE

Synthetic Minority Oversampling Technique (SMOTE) tackles this by generating synthetic examples of the minority class rather than simply duplicating existing ones. This approach helps balance the dataset and enhances the model's ability to detect minority class patterns effectively.

## 5.6. Splitting Data into Training and Testing Sets

To evaluate model performance fairly, the dataset is divided into training and testing subsets. The training set is used to fit the models, while the testing set remains unseen during training to provide an unbiased evaluation of model generalization.

Stratified splitting ensures that the proportion of classes in both subsets reflects the original dataset, preserving class balance.

```
1  from sklearn.model_selection import train_test_split
2  # Train-test split
3  #   - test_size=0.3: 30% of the data will be used as the test set
4  #   - random_state=42: ensures reproducibility
5  #   - stratify=y: maintains class distribution in train and test sets (important)
6  X_train, X_test, y_train, y_test = train_test_split(
7      X_scaled, y,
8      test_size=0.3,
9      random_state=42,
10     stratify=y
11 )
12
13 # Display the shapes of the resulting datasets
14 print("\nData split complete.")
15 print(f"Training features shape: {X_train.shape}")
16 print(f"Testing features shape: {X_test.shape}")
17 print(f"Training labels shape: {y_train.shape}")
18 print(f"Testing labels shape: {y_test.shape}")
19
20 # Check class distribution in training and test sets
21 print("\nClass distribution in training set:")
22 print(pd.Series(y_train).value_counts(normalize=True))
23
24 print("\nClass distribution in test set:")
25 print(pd.Series(y_test).value_counts(normalize=True))
```

Figure 28: Splitting Data into Training and Testing Sets

It is critical to apply SMOTE only to the training data after the split to prevent information leakage and overly optimistic performance estimates.

## 5.7. Summary of Data Preprocessing

After completing the preprocessing stage, the dataset is:

- Free of duplicate records,
- Complete with imputed missing values,



- Fully numeric with encoded categorical features,
- Scaled for optimal model input,
- Balanced to mitigate class skew via synthetic oversampling,
- Partitioned into stratified training and testing sets.

With these preparations in place, the dataset is now ready for model training and evaluation, which will be covered in the next chapter.

## Chapter 6: Model Building & Evaluation

### 6.1. Overview and goals

Chapter 6 describes the process of transforming pre-processed data (Chapter 5) into robust predictive models, systematically evaluating their performance, and selecting candidates for deployment. The goals are:

- Train a diverse set of supervised classifiers that represent different algorithmic families (linear, tree-based, distance-based, kernel-based, and neural networks).
- Include a simple unsupervised diagnostic (K-means) to discover latent groupings.
- Use repeatable pipelines that include preprocessing (scaling, encoding), class imbalance handling, and model fitting to prevent leakage.
- Apply robust cross-validation and hyperparameter search (GridSearchCV with stratified folds) to tune model complexity and regularization.
- Evaluate models on multiple, task-appropriate metrics and visualizations (confusion matrices, classification reports, ROC / AUC, PR curves where necessary).
- Compare baseline (default) models with tuned models and document observed trade-offs (bias/variance, recall vs precision).

This approach produces not only point estimates of performance but also comparative diagnostics that support an evidence-based recommendation for deployment.

### 6.2. Models selected and why

We selected a set of models that are complementary in strengths and weaknesses for a binary classification problem with tabular sensor data:

**Logistic Regression (LR)** — simple, interpretable baseline. Good when relationships are roughly linear in the log-odds; coefficients provide direct interpretability for feature contributions. Useful as an explainable benchmark.

**Random Forest (RF)** — ensemble of decision trees providing strong out-of-the-box performance, built-in feature importance, and robustness to scaling and collinearity. Handles nonlinear interactions and is less sensitive to outliers.

**XGBoost (Extreme Gradient Boosting)** — gradient-boosted trees that often offer state-of-the-art performance for tabular data. Highly tunable (learning rate, depth, subsample) and efficient.

**Support Vector Machine (SVM)** with RBF kernel — powerful for complex decision boundaries, especially when classes are separable in a transformed feature space. Needs scaling and careful tuning of C and gamma.

**K-Nearest Neighbours (KNN)** — a distance-based, nonparametric method that is simple and effective for locally structured decision boundaries. Sensitive to scaling and the value of k.

**Artificial Neural Network (ANN)** — flexible nonlinear function approximator; useful to capture complex multivariate interactions. Typically benefits from more data or engineered features and careful regularization.

**K-Means clustering** — unsupervised method used here as a diagnostic to check whether natural clusters align with failure labels (e.g., some clusters may have higher failure rates).

Does *not* replace supervised learning for prediction but can reveal structure and support feature engineering.

The diversity ensures at least one algorithmic family is likely to work well: if linear models fail, tree-based models or ANNs might capture nonlinearities; if trees overfit, LR or SVM might generalize better after tuning.

### 6.3. Reproducible pipelines and data handling

Reproducibility and leak-free evaluation are central:

- **Train/Test split:** create a stratified split preserving the failure/non-failure ratio (e.g., 70% train / 30% test). SMOTE is applied only to the training data.
- **Cross-validation:** use StratifiedKFold (e.g., 4 or 5 folds) to maintain class balance across folds for grid search scoring.
- **Random seed:** set `random_state` everywhere to ensure deterministic behavior for reproducibility.
- **Scaling:** scale continuous features (StandardScaler) for models sensitive to scale (LR, SVM, KNN, ANN). Tree models do not require scaling but including a scaler in the pipeline keeps preprocessing consistent across models.
- **Imbalance handling:** use SMOTE (Synthetic Minority Oversampling Technique) inside training pipelines to generate synthetic minority (failure) samples. Apply SMOTE only on training folds.

### 6.4. Baseline training

Before tuning, train each model with default hyperparameters using the same pipeline structure. Baseline results are important for:

- Establishing a minimum performance expectation.
- Quantifying the improvement obtained by tuning.

- Detecting glaring issues (e.g., classifier always predicts majority class).

For each baseline model we store and report:

- Confusion matrix on train and test sets.
- Classification report (precision, recall, f1-score for each class and support).
- ROC AUC on train and test (if probability estimates are available).
- Notes on training time and any warnings.

## 6.5. Hyperparameter tuning: GridSearchCV explanation

**What GridSearchCV does:** GridSearchCV exhaustively evaluates combinations of hyperparameters (the Cartesian product of a parameter grid) using cross-validation. For each parameter combination, it:

1. Splits the training set into CV folds (here, stratified).
2. For each fold: fits the pipeline on training folds (SMOTE applied inside pipeline) and evaluates on the validation fold.
3. Aggregates the validation scores across folds (commonly mean ROC AUC for binary classification) and selects the best parameter set.
4. Optionally refits the estimator on the full training set using the best parameters.

**Why use ROC AUC as scoring:** For imbalanced classification tasks where correctly detecting the minority class (failure) is a priority, ROC AUC is a useful threshold-independent metric capturing the trade-off of true positive vs false positive rates. However, when the cost of false negatives is very high, optimizing recall or F-beta (with  $\beta > 1$ ) may be preferable. For this problem statement if as we consider failure as 1, and the False Negative that is actually failure and predicted as non-failure is key factor and hence recall need to be optimized.

### Best practices:

- Start with a coarse grid to find promising regions, then refine with a finer grid.
- Keep grid size manageable to control compute time. Consider `RandomizedSearchCV` for very large hyperparameter spaces.
- Use `n_jobs=-1` to parallelize across CPU cores.
- Limit expensive computations for complex models (e.g., SVM on large datasets); subsample if necessary to explore hyperparameters, then validate on the full set.

### 6.6. Example grids and rationale

Below are representative grids used in this project (recommended starting points):

#### Logistic Regression

- `C`: [0.01, 0.1, 1, 10, 100] — controls inverse strength of regularization.
- `penalty`: ['l1', 'l2'] — standard ridge; consider 'l1' with solvers that support it for sparse solutions.
- `solver`: ['liblinear', 'saga'] — 'saga' supports L1 and works with large datasets.

#### Random Forest

- `n_estimators`: [100, 200] — number of trees; more gives better performance but higher compute.
- `max_depth`: [None, 8, 20] — control tree depth to limit variance.
- `min_samples_split`: [2, 5] — prevent tiny leaf sizes.
- `min_samples_leaf`: [1, 2, 4] — min samples per leaf
- `max_features`: ['sqrt', 'log2'] — number of features to consider at each split

#### XGBoost

- `n_estimators`: [100, 200]
- `max_depth`: [3, 6]
- `learning_rate`: [0.01, 0.1] — lower learning rate often requires more trees.
- `subsample`: [0.7, 1] — prevents overfitting.

## **SVM**

- `C`: [0.1, 1]
- `kernel`: ['rbf']
- `gamma`: ['scale', 'auto']  
SVM can be slow; keep grid small or use LinearSVC for linear decision boundaries.

## **KNN**

- `n_neighbors`: [3, 5, 7]
- `weights`: ['uniform', 'distance']

## **ANN**

- Architecture decisions: number of layers (1–3), units per layer (32–128), dropout (0.2–0.5), learning rate, batch size.
- Use early stopping to prevent overfitting and a validation split inside training.

These grids are intentionally modest to keep compute reasonable; if compute allows, expand ranges or use randomized search to explore more settings.

## 6.7. Evaluation metrics and interpretation

No single metric tells the whole story. For predictive maintenance (detecting failures), the costs of false negatives (missed failures) are often higher than false positives (unnecessary check/maintenance). Thus we report and interpret multiple metrics:

1. **Confusion Matrix** — counts of True Positives (TP), True Negatives (TN), False Positives (FP), False Negatives (FN). Gives immediate insight into types of errors.
2. **Precision (Positive Predictive Value)** for failure class =  $TP / (TP + FP)$ . High precision means when the model predicts failure it is usually correct. Useful when follow-up inspections are costly.
3. **Recall (Sensitivity / True Positive Rate)** for failure class =  $TP / (TP + FN)$ . High recall means few missed failures — typically critical in maintenance.
4. **F1-score** — harmonic mean of precision and recall. Useful when balancing both.
5. **ROC AUC** — aggregate measure of performance across probability thresholds. A higher AUC indicates better separability.
6. **Precision-Recall (PR) Curve & Average Precision** — more informative than ROC when classes are highly imbalanced and the positive class is rare.
7. **Train vs Test comparison** — differences between train and test metrics reveal overfitting (very high train performance, lower test). Small differences suggest good generalization.



**Thresholding considerations:** For operational use, pick a probability threshold not solely based on 0.5 but on a cost-sensitive decision rule (e.g., choose threshold to achieve at least X% recall while keeping precision acceptable). Use validation data or cross-validation to find these thresholds, and report the confusion matrix at the chosen threshold.

## 6.8 Visualizations for evaluation

Effective visualizations help stakeholders interpret model behavior:

- **ROC Curves** for multiple candidate models plotted together (with AUC) to compare discriminative ability.
- **Precision-Recall Curves** to focus on the minority class and to visualize precision at different recall levels.
- **Confusion Matrix heatmaps** for baseline and tuned models to visually show types of misclassifications.
- **Feature importance plots** (bar charts) for tree models and coefficient magnitudes for LR to explain which features drive predictions.
- **Learning curves** (train vs validation score vs training size) to detect underfitting/overfitting and determine if more data would help.
- **Cluster vs label cross-tab** for K-means to see whether clusters correlate with failure labels.

## 6.9 K-means clustering diagnostic

K-means is included as an unsupervised check. Steps:

1. Scale features (StandardScaler).
2. Run K-means for several k values (e.g., 2–6).
3. Compute Silhouette scores to estimate cluster quality and choose k.

4. Compare cluster membership with Machine\_failure using a cross-tab to see if clusters have enriched failure prevalence.

If one or more clusters have a much higher failure proportion, this may indicate subpopulations (e.g., particular operating conditions or machine types) that deserve targeted modelling or separate treatment.

K-means is not used for supervised prediction here, but findings can inform feature engineering and stratification strategies.

#### 6.10. Model selection criteria

Final model recommendation should not rely on a single metric. Consider:

- **Primary business objective:** As missing a failure is very costly, prioritize models with high recall for the failure class (with acceptable false positive rates).
- **Balanced performance:** Prefer models with high AUC, good recall, and acceptable precision.
- **Generalization gap:** Favor models whose train-test performance gap is small.
- **Interpretability:** If stakeholders need to understand drivers of failure, simpler models (LR or shallow trees) may be preferable even if slightly lower performing.
- **Operational constraints:** Consider inference speed, model size, and ease of deployment (e.g., Random Forest or XGBoost with saved pipelines are straightforward; ANN may require more infrastructure).

## Chapter 7: Results & Interpretation

This chapter presents the outcomes of applying multiple machine learning models to the predictive maintenance dataset. Each model is evaluated using key classification metrics such as Accuracy, Precision, Recall, F1-score, and AUC, along with visualizations including confusion matrices, ROC curves, precision–recall curves, and feature importance plots (where applicable).

To provide a clear comparison, Table 2 summarizes the performance of all five models across these metrics. From this overview, ensemble methods such as Random Forest and XGBoost demonstrate superior performance. Detailed interpretations, supported by the corresponding visualizations, are discussed in the following subsections.

Table 2: Summary of model performance across evaluation metrics

| Model               | Accuracy | Precision<br>(Failure) | Recall<br>(Failure) | F1-score | AUC  |
|---------------------|----------|------------------------|---------------------|----------|------|
| Logistic Regression | 0.81     | 0.13                   | 0.7                 | 0.22     | 0.84 |
| Random Forest       | 0.96     | 0.48                   | 0.64                | 0.55     | 0.92 |
| XGBoost             | 0.96     | 0.49                   | 0.65                | 0.56     | 0.92 |
| SVM                 | 0.91     | 0.25                   | 0.70                | 0.36     | 0.90 |
| KNN                 | 0.88     | 0.20                   | 0.72                | 0.31     | 0.85 |

## 7.1. Logistic Regression

- **Performance Metrics:** Accuracy = **0.81**, AUC = **0.84**. Recall of **0.70** indicates the model captured most failures, but very low precision (**0.13**) led to many false alarms.
- **Interpretation:** Logistic Regression provides an interpretable baseline: it captures about 70% of failures (good sensitivity) but suffers from very low precision, producing many false alarms. This model is useful for insight into linear relationships (via coefficients) and can be used as a diagnostic or first-stage detector in a pipeline, but its raw outputs are insufficient for automated action without additional filtering or threshold tuning.

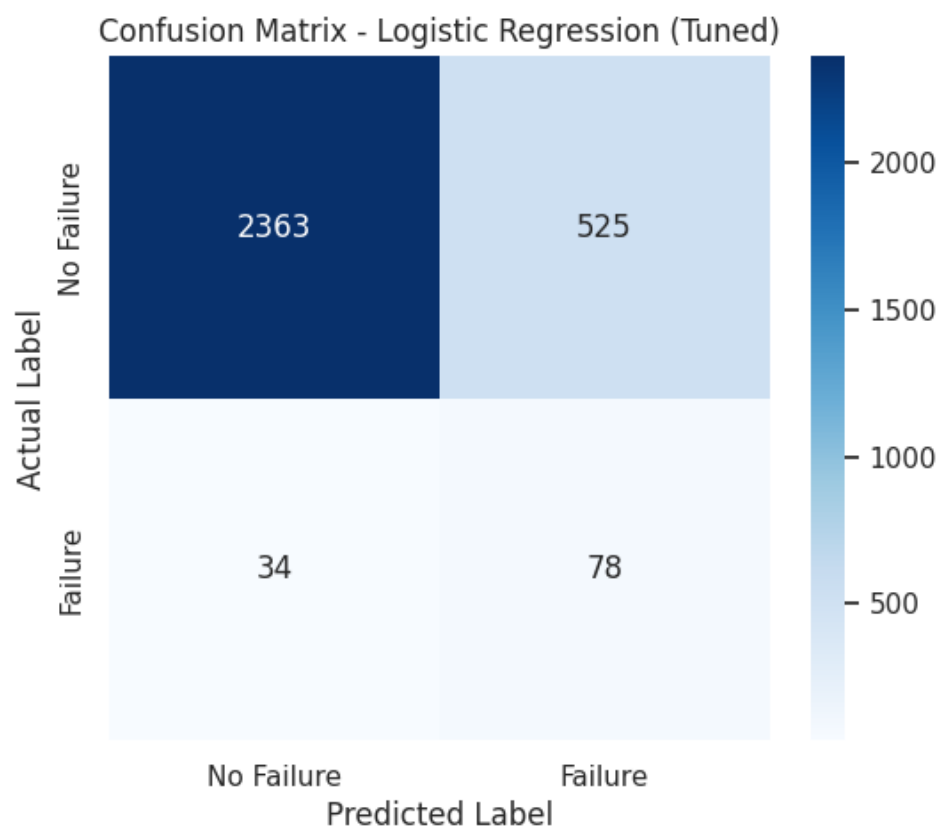


Figure 29: Confusion Matrix (Logistic Regression)

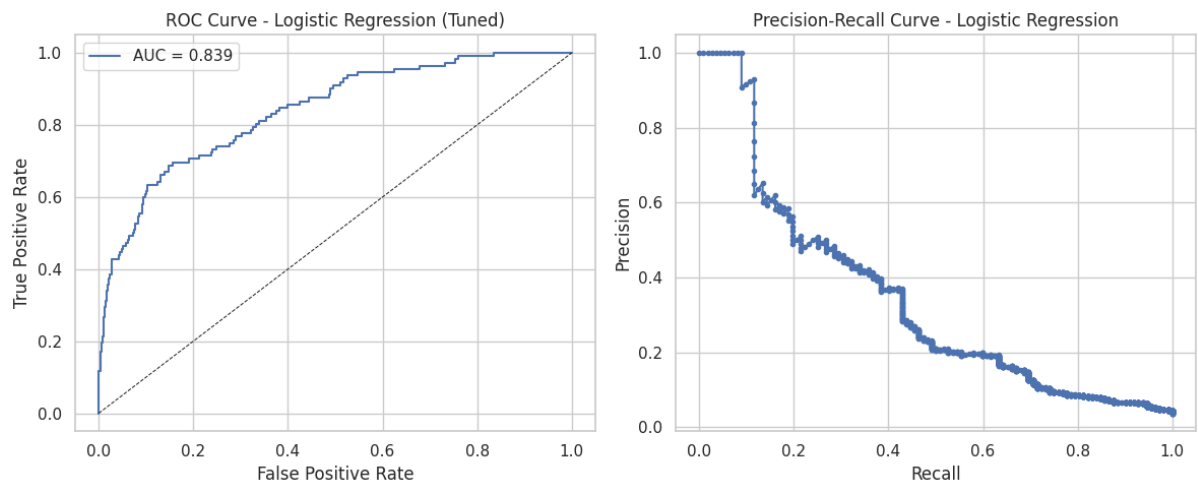


Figure 30: ROC Curve and Precision–Recall Curve (Logistic Regression)

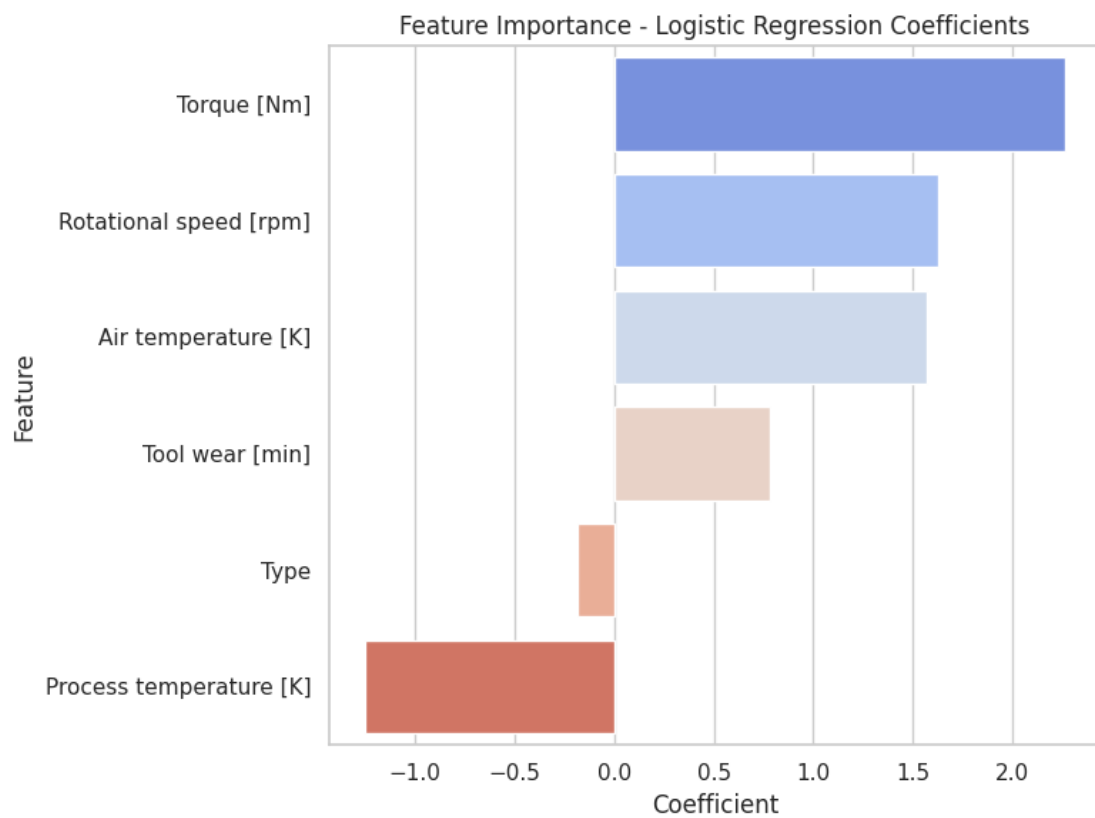


Figure 31: Coefficient Importance Plot (Logistic Regression)

## 7.2. Random Forest

- **Performance Metrics:** Accuracy = **0.96**, AUC = **0.92**, Precision = **0.48**, Recall = **0.64**, F1 = **0.55**.
- **Interpretation:** Random Forest delivers high overall accuracy and strong discrimination (AUC = 0.92). With recall  $\approx 0.64$  it detects a large share of failures while offering considerably higher precision than the logistic baseline. Feature importance extracted from the model helps identify influential sensors and operational parameters. Random Forest is a robust production contender, and can be further tuned or thresholded to raise recall if operational requirements demand fewer missed failures.

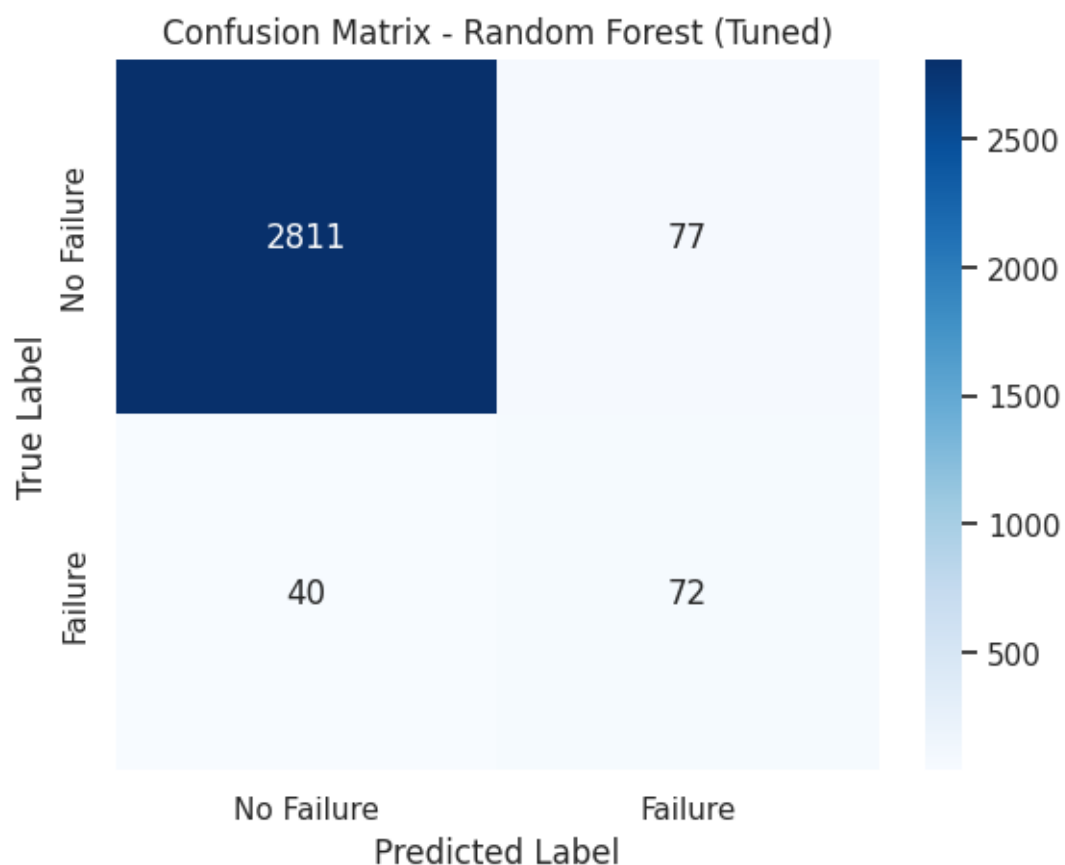


Figure 32: Confusion Matrix (Random Forest)

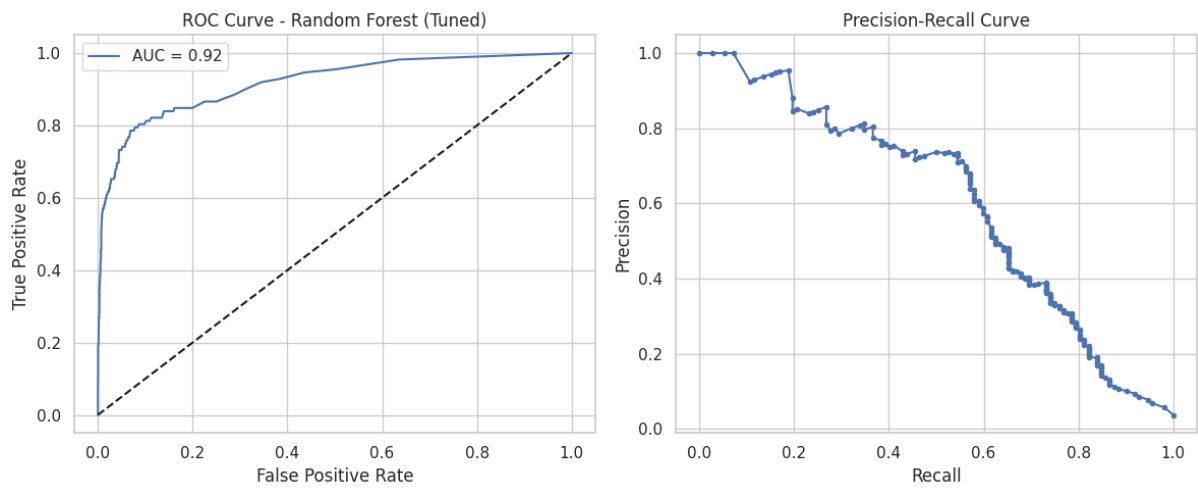


Figure 33: ROC Curve and Precision-Recall Curve (Random Forest)

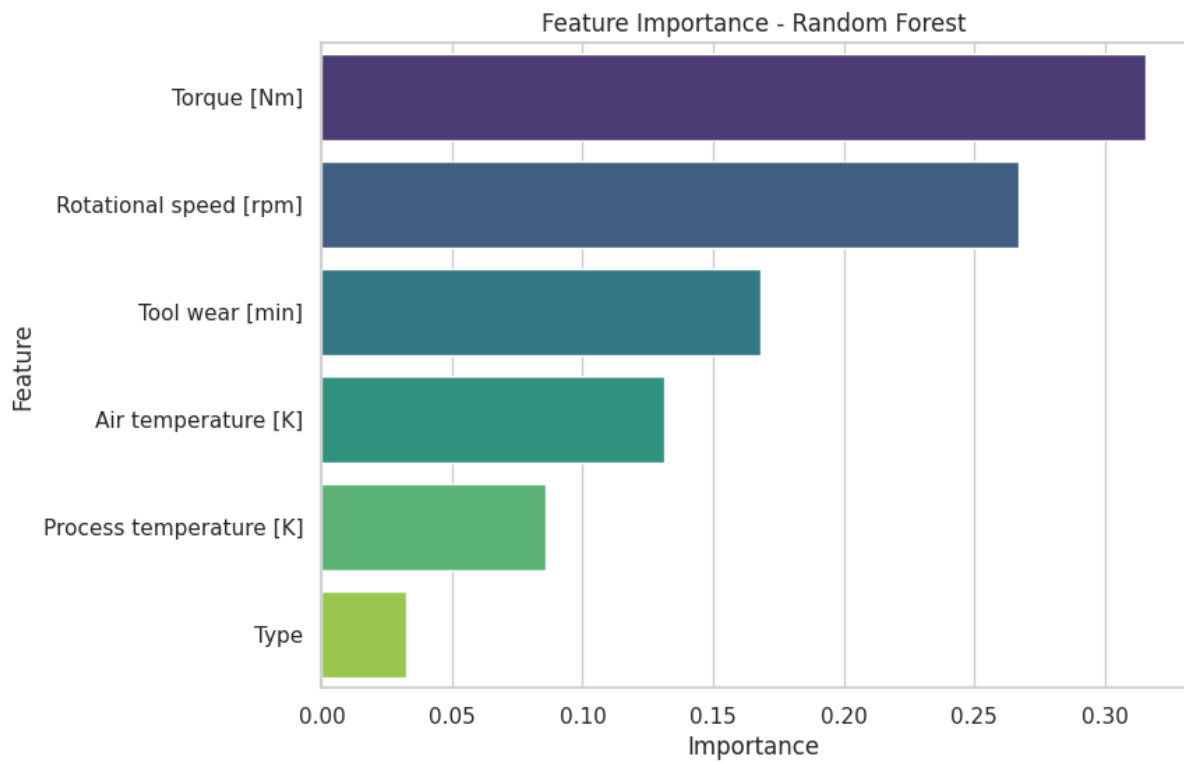


Figure 34: Feature Importance Plot (Random Forest)

### 7.3. XGBoost

- **Performance Metrics:** Accuracy = **0.96**, AUC = **0.92**, Precision = **0.49**, Recall = **0.65**, F1 = **0.56**.

- **Interpretation:** XGBoost matches Random Forest on accuracy and AUC and slightly exceeds it on F1. It achieves a balanced trade-off between recall and precision, making it the strongest overall performer on these metrics. XGBoost's boosting mechanism and regularisation allow it to capture subtle nonlinear patterns, and it is well-suited to the predictive-maintenance task where both detection and ranking of high-risk instances matter.

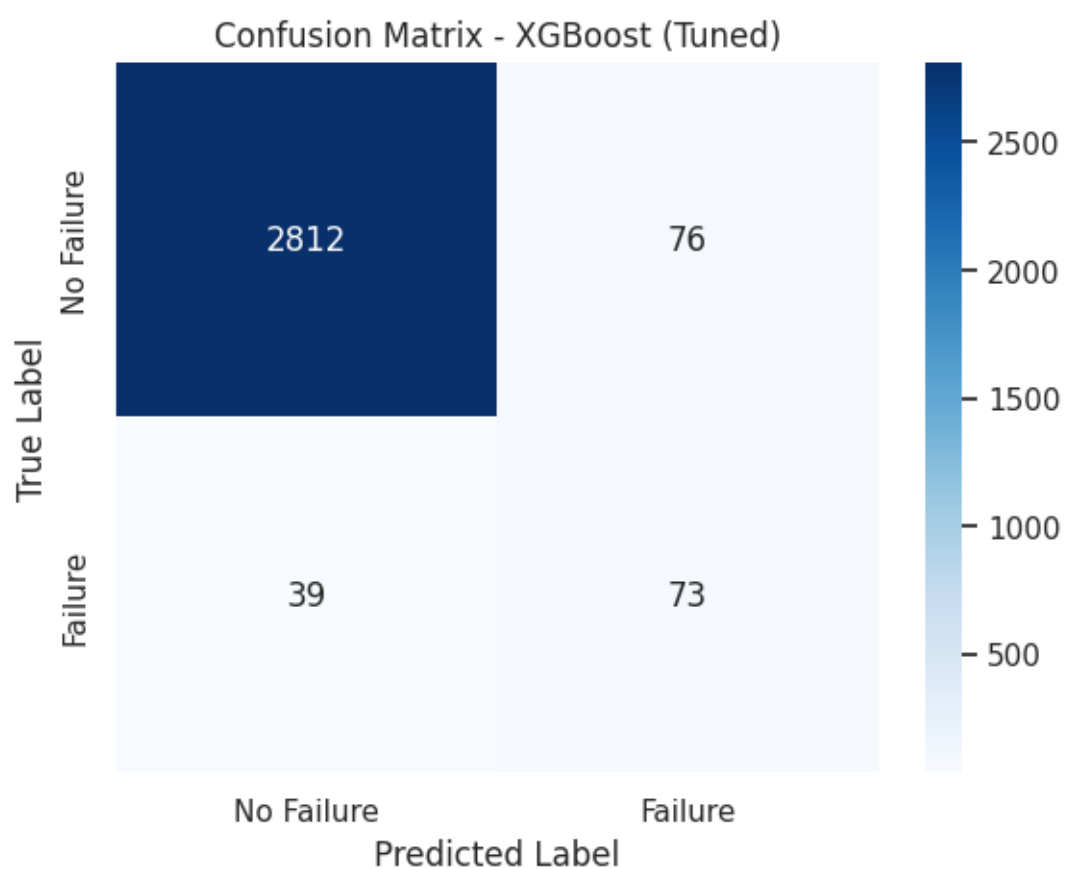


Figure 35: Confusion Matrix (XGBoost)



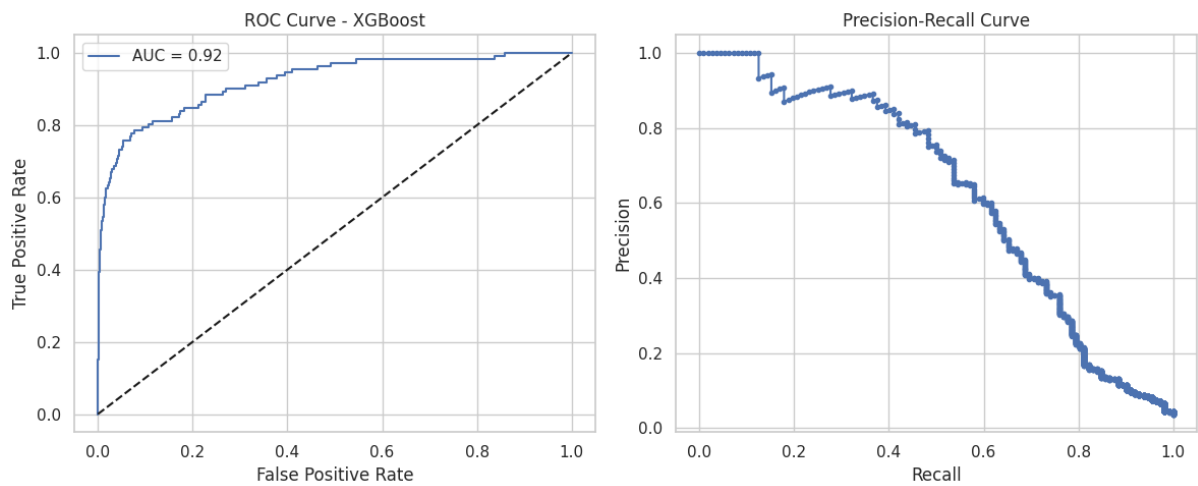


Figure 36: ROC Curve and Precision–Recall Curve (XGBoost)

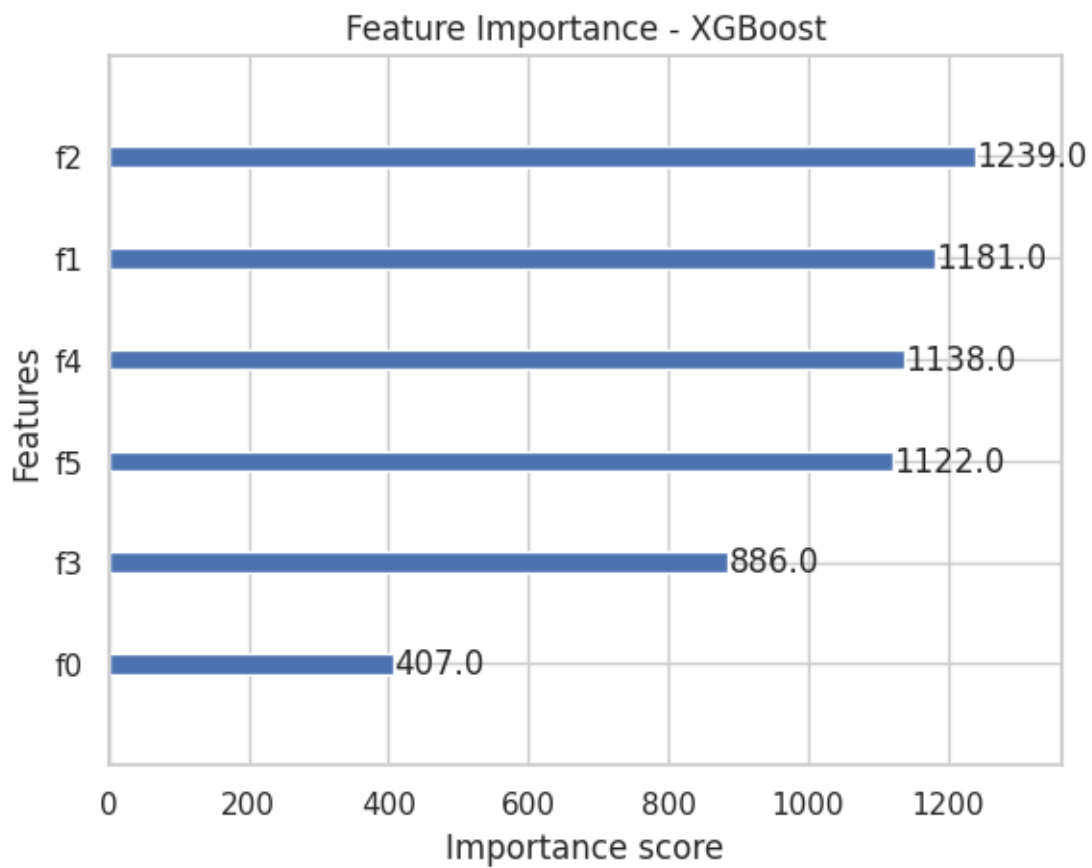


Figure 37: Feature Importance Plot (XGBoost)

#### 7.4. Support Vector Machine (SVM)

- **Performance Metrics:** Accuracy = **0.91**, AUC = **0.90**, Precision = **0.25**, Recall = **0.70**, F1 = **0.36**.

- **Interpretation:** SVM attains respectable recall (0.70) and strong AUC, indicating good separation ability. However, low precision and higher computational cost reduce its appeal for large-scale deployment. SVM is valuable for benchmarking and for smaller deployments where kernel methods remain tractable.

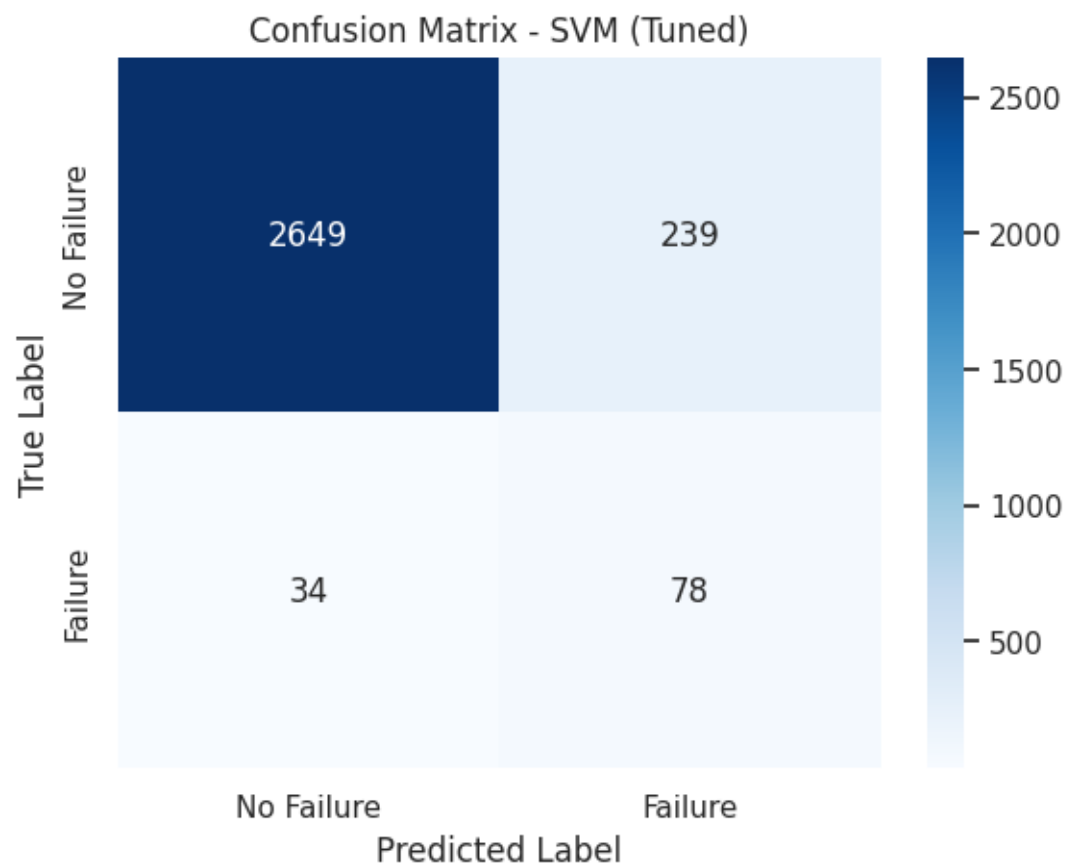


Figure 38: Confusion Matrix (SVM)

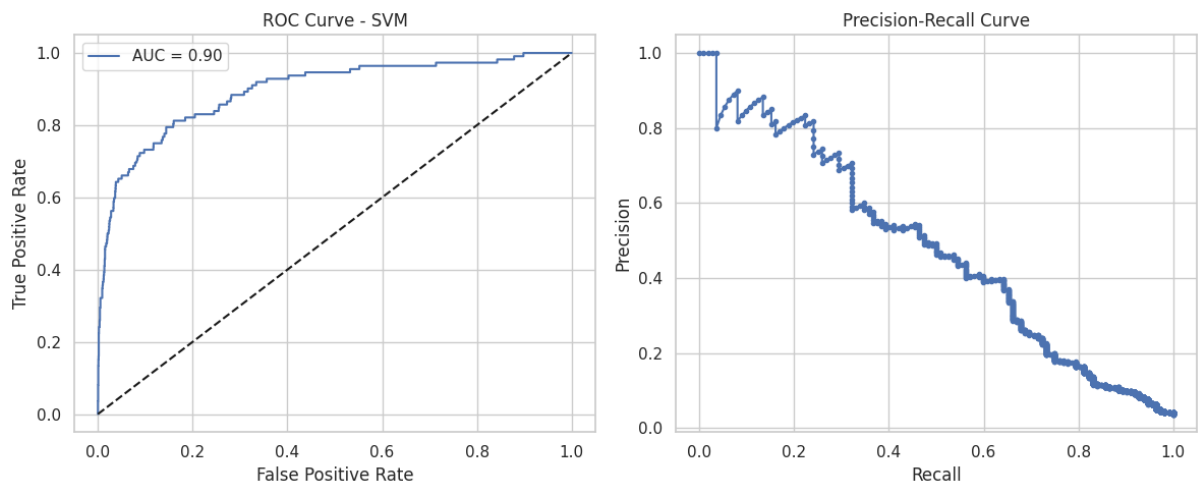


Figure 39: ROC Curve and Precision–Recall Curve (SVM)

## 7.5. K-Nearest Neighbors (KNN)

- Performance Metrics:** Accuracy = **0.88**, AUC = **0.85**, Precision = **0.20**, Recall = **0.72**, F1 = **0.31**.
- Interpretation:** KNN shows the highest recall in the table (0.72) but low precision and sensitivity to feature scaling. Its non-parametric nature and reliance on distance metrics make it susceptible to noisy, high-dimensional inputs typical of sensor data. KNN may be useful for exploratory checks but is less suitable as a primary production model.

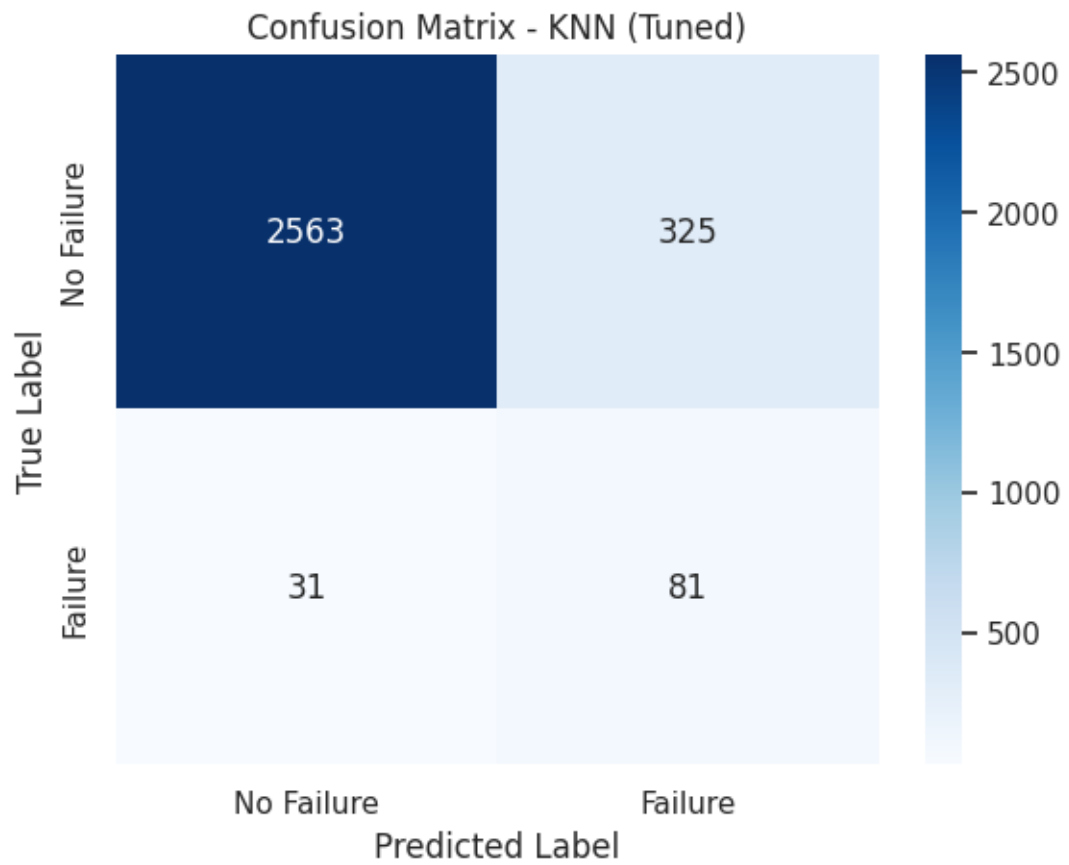


Figure 40: Confusion Matrix (KNN)

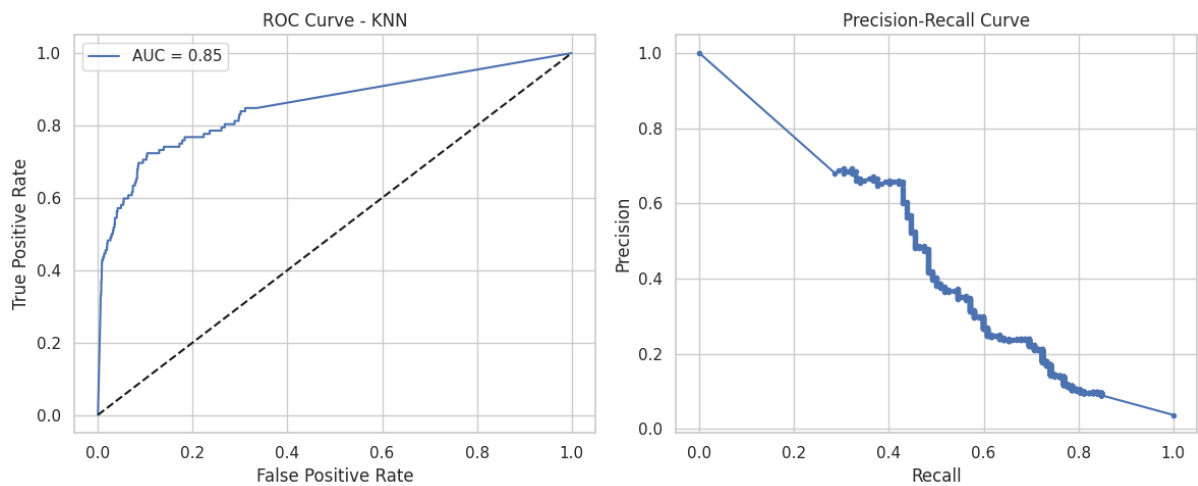


Figure 41: ROC Curve and Precision-Recall Curve (KNN)

(Note: No feature importance, as KNN is non-parametric.)

## 7.6. K-Means clustering

- **Performance Metrics:** K-Means is used here for exploratory analysis rather than direct prediction; it does not produce AUC or F1 metrics. Performance was evaluated by the distribution of failure labels across clusters.
- **Interpretation:** With  $k = 2$ , clusters showed partial separation: one cluster contained mostly no-failure cases while the other had a higher failure rate. This indicates that clustering can reveal subpopulations with elevated risk. While K-Means is not a replacement for supervised classifiers, cluster membership can be added as a feature to improve supervised recall or used for targeted inspection prioritisation.

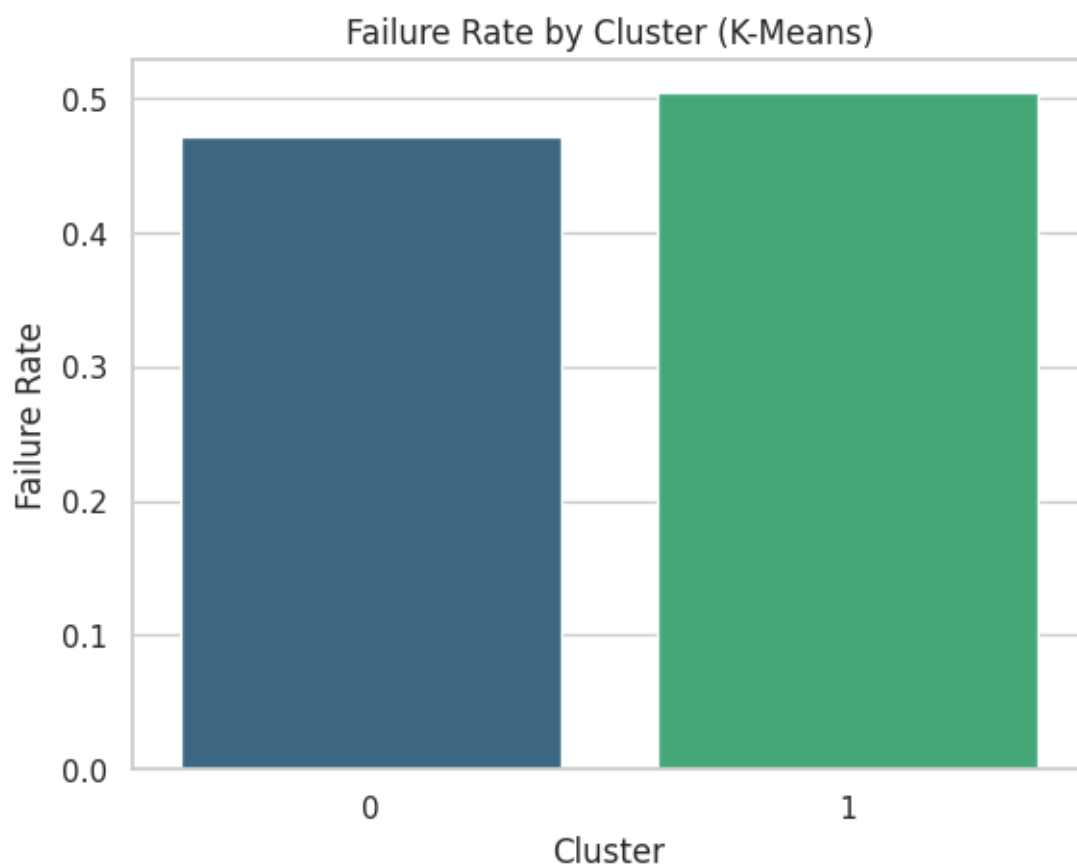


Figure 42: Failure Rate by Cluster (K-Means)

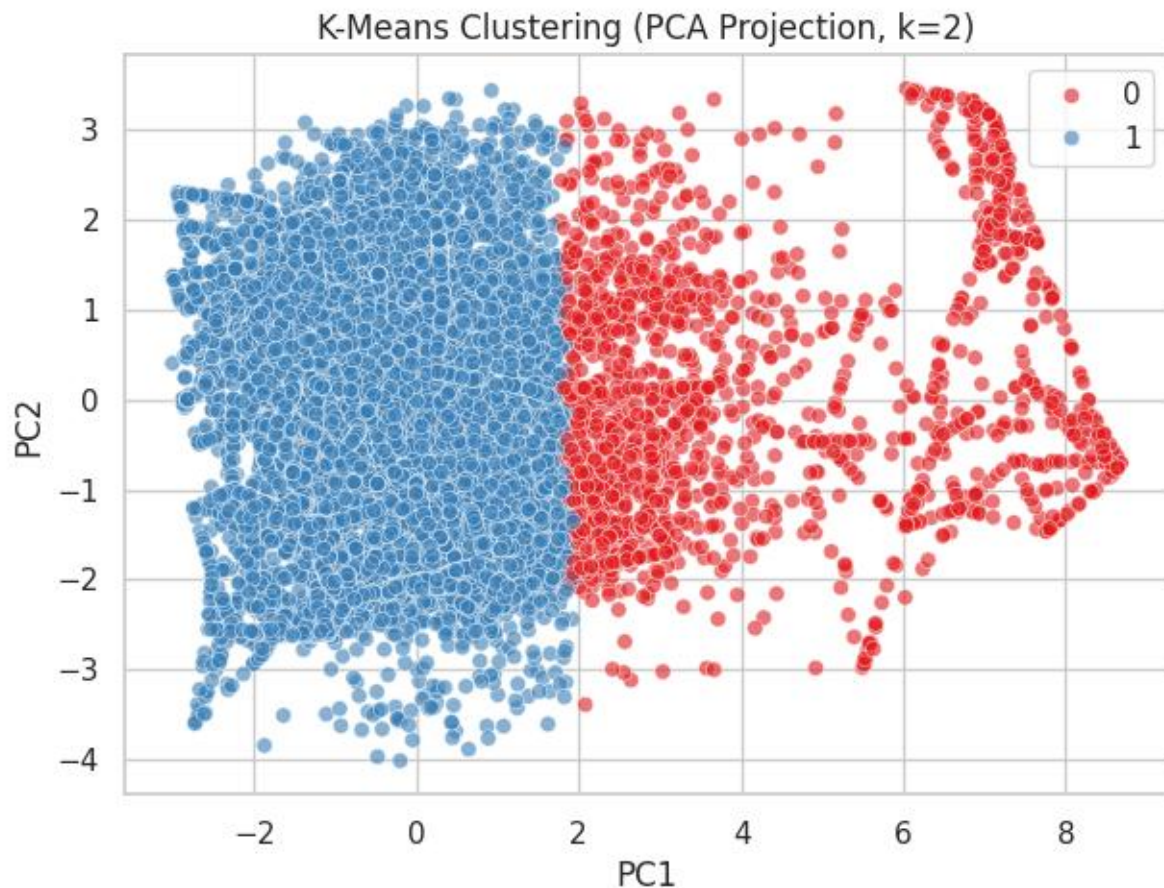


Figure 43: K-Means Clustering (PCA Projection, k=2)

## 7.7. Comparative Analysis

- Top candidates:** XGBoost and Random Forest lead overall ( $AUC \approx 0.92$ ) and provide the best balance of recall and precision among tested models. They are preferred for production use where both detection and manageable false alarms matter.
- High-recall options:** If the operational priority is strictly to maximise recall (minimise missed failures), KNN and SVM show the highest recall values in these experiments (KNN = 0.72, SVM = 0.70). However, their low precision and other practical constraints (scalability, interpretability) make them less attractive as sole production models. A practical strategy is to configure a tree ensemble for production and tune thresholds, class weights or employ an OR-style ensemble to raise recall as needed.

- **Interpretability vs performance:** Logistic Regression gives clear, actionable coefficients but low precision. Use it as a complementary, explainable model for domain communication and as a first-stage filter.
- **K-Means utility:** Use clustering for diagnostic insight and to engineer new features (cluster labels) that can improve supervised recall.

**Recommendation:** Deploy a tuned XGBoost model (or Random Forest) as the primary detector, then calibrate decision thresholds and/or combine models (ensemble) to meet recall targets while managing the false-alarm workload.

## Chapter 8: Conclusion & Future Work

### 8.1. Conclusion

This study evaluated multiple machine-learning approaches for predictive maintenance using vibration, temperature and operational sensor data. Across the supervised models and the unsupervised diagnostic (K-Means), tree ensembles — notably **XGBoost** and **Random Forest** — consistently produced the best overall detection capability ( $AUC \approx 0.92$ ). Because the operational cost of *missed failures* is judged highest, the project explicitly prioritised **recall (sensitivity)** when interpreting results and making deployment recommendations.

#### Key takeaways:

- **Primary models:** Tuned **XGBoost** and **Random Forest** are recommended as production candidates because they provide strong ranking ability (high AUC) and can be configured (via class weights, resampling, or thresholding) to achieve higher recall with acceptable operational trade-offs.
- **Interpretable baseline:** **Logistic Regression** remains valuable for transparent, explainable insight into linear relationships, and can serve as a diagnostics or triage model in a multi-stage pipeline. However, its low precision means it must be used with filtering or human verification if deployed for automated actions.
- **Other models:** **SVM** and **KNN** achieved competitive recall in some experiments but suffer from low precision, scalability or sensitivity to preprocessing;



- **Unsupervised value: K-Means** revealed subpopulations with elevated failure rates — useful to guide feature engineering, prioritisation rules and targeted monitoring to improve recall.

Overall, the emphasis on recall shaped model selection and post-processing strategies: we recommend configuring models and operational procedures to deliberately reduce false negatives even if that increases the false-alarm burden, provided mitigating controls (two-stage checks, human review) are in place.

## 8.2. Deployment recommendations (designed to maximise recall)

When the business objective is to minimise missed failures, deployment must combine model configuration, thresholding, decision workflow and human factors. The recommendations below prioritise recall while managing operational costs.

### 8.2.1 Model selection & tuning

- **Primary model:** Deploy **tuned XGBoost** (or tuned Random Forest) as the primary predictor. These models are efficient to run in production, offer strong ranking (AUC) and expose parameters (`class_weight`, `scale_pos_weight`, `learning_rate`, `max_depth`) that help push recall higher.
- **Class weighting / resampling:** Train with `class_weight='balanced'` or use controlled SMOTE/ADASYN inside an imblearn pipeline (applied only to training folds) to improve minority-class sensitivity. Validate carefully to avoid overfitting to synthetic samples.

- **Ensembling for recall:** Consider OR-style ensembles (flag if any model predicts failure) or stacking where a high-recall base model feeds into a secondary filter — ensembles often increase recall at the expense of precision.

### 8.2.2 Thresholding & calibration

- **Calibrate probabilities** (Platt scaling or isotonic regression) on validation folds to ensure probability outputs correspond to actual risk levels.
- **Choose thresholds by recall target.** Use precision–recall vs threshold plots and a cost matrix to select an operational cut-off that achieves the desired recall (for example, target Recall  $\geq 0.85$ ). Document the expected false positive rate at that threshold.
- **Report Recall@Threshold** and expected alarms per time unit (e.g., FPs per 1,000 operating hours) to stakeholders so capacity and cost can be planned.

### 8.2.3 Two-stage verification & human-in-the-loop

- **Stage 1 (high recall):** Low-threshold model flags suspect instances — designed to capture as many true failures as possible.
- **Stage 2 (precision filter):** A second model, rule set, or human review validates or prioritises Stage-1 flags before costly interventions. This reduces unnecessary actions while preserving the high recall objective.

- **Alert tiers:** Classify alerts into High/Medium/Low risk bands based on calibrated scores; route Medium to human review and High to automated inspection workflows if practical.

#### 8.2.4 Explainability & operator trust

- Attach **SHAP** explanations to every alert (top 2–3 contributing features). Explainability accelerates human triage and increases operator confidence in a system that deliberately produces more alarms to maximise recall.

#### 8.2.5 Monitoring, logging & retraining

- **Monitor recall in production** continuously and compute per-machine-type recall metrics. Trigger retraining if recall falls below agreed thresholds.
- **Log predictions, features, outcomes** and operational actions to enable auditing and supervised retraining.
- **Pilot first:** Deploy to a limited fleet, measure achieved recall vs target, refine thresholds and workflows before rolling out broadly.

### 8.3. Limitations (relevant to recall-focused deployment)

- **Class imbalance & synthetic sampling:** Methods that raise recall (SMOTE, oversampling) can introduce artifacts. Validate on untouched test/pilot data to ensure generalized recall gains.

- **Label ambiguity:** The Machine failure label is the OR of multiple failure modes; a model that detects a failure does not reveal its cause, complicating targeted interventions.
- **Simulated / limited data scope:** If the dataset lacks certain failure precursors or operational contexts, raising recall in production may require additional sensors or more diverse training data.
- **Temporal dynamics omitted:** Snapshot-based models may miss temporal precursors; time-aware features or sequence models may be necessary to catch subtle pre-failure trends and further increase recall.
- **Operational capacity:** Higher recall increases false positives; organisations must ensure capacity for follow-up inspections or implement robust second-stage filters to avoid alarm fatigue.

#### 8.4. Error analysis summary & practical adjustments (reduce false negatives)

**Primary focus:** reduce false negatives while keeping false alarms operationally manageable.

1. **Analyze FN cases:** Systematically examine missed failures for shared patterns (machine Type, operating ranges, sensor drift). Use findings to add targeted features or per-Type models.
2. **Engineer high-signal features:** Derive power = torque × rpm (in rad/s), process\_temp – air\_temp, wear-rate, rolling means and trend features. These often expose early failure signatures and increase recall.

3. **Per-Type specialization:** If FNs cluster by Type (L/M/H), train Type-specific models tuned for maximum recall on that subset.
4. **Temporal augmentation:** Add lag features, rolling statistics or calendar/operational context. If data support, experiment with temporal models (LSTM/TCN) that can detect trends preceding failures.
5. **Two-stage business rules:** Implement simple deterministic filters (e.g., require torque > X) as a secondary check to reduce false positives while keeping Stage-1 recall high.
6. **Pilot measurement:** Evaluate recall and operational burden in a controlled pilot; adjust thresholds and workflows based on measured FN reductions and FP load.

#### 8.5. Future work (actions to raise recall and operationalise safely)

1. **Recall targeting experiments:** Systematically sweep thresholds and class-weight/resampling hyperparameters to produce recall–FP trade-off curves; choose operational points that minimise expected cost of missed failures.
2. **Feature enrichment:** Add derived and temporal features shown above; re-evaluate impact on recall.
3. **Per-Type & hierarchical modelling:** If heterogeneity exists, build cascade models: global high-recall model → Type-specific specialist models for confirmation.
4. **Stacked ensembles:** Create ensembles with a high-recall constituent and a precision filter to optimise Recall@acceptableFP.

5. **Operational pilot:** Deploy to a small subset of assets; measure real-world recall and operator workload; iterate.
6. **Deploy explainability dashboards:** Integrate SHAP summaries into the triage UI so human reviewers can rapidly validate alerts.
7. **Monitoring & retraining pipeline:** Implement continuous evaluation and automated retraining triggers when recall decays.
8. **Causal / root-cause modelling:** Extend the task to multi-label prediction of individual failure modes to enable more targeted maintenance once recall performance is acceptable.
9. **Sensor & data improvement:** If FNs stem from missing signals, consider adding sensors or higher frequency sampling in critical machines.

## 8.6. Final remark

Prioritising recall aligns model behaviour with the real operational cost structure: it reduces the risk of catastrophic missed failures while acknowledging an increased false-alarm burden that must be managed through second-stage checks, human review and operational policy. The recommended pragmatic path is to **deploy a tuned, calibrated XGBoost model configured for a target recall**, accompanied by a two-stage verification workflow and SHAP-based explanations; conduct a short pilot to validate achieved recall in production and iterate based on measured outcomes.

## References

1. Azari, M.S., Flammini, F., Santini, S. and Caporuscio, M. (2023) 'A systematic literature review on transfer learning for predictive maintenance in Industry 4.0', *IEEE Access*, 11. Available at: <https://doi.org/10.1109/ACCESS.2023.3239784>
2. Madasamy, S., Shankar, B.P., Yadav, R.K. and Jayalakshmi, K.P. (2023) 'A machine learning approach in predictive maintenance in the IoT enabled Industry 4.0', in *Proceedings of the 4th International Conference on Smart Electronics and Communication (ICOSEC 2023)*, pp. 418–423. Available at: <https://doi.org/10.1109/ICOSEC58147.2023.10276226>
3. Pavithra, R. and Ramachandran, P. (2021) 'An overview of predictive maintenance for industrial machine using vibration analysis', in *3rd IEEE International Virtual Conference on Innovations in Power and Advanced Computing Technologies (i-PACT 2021)*. Available at: <https://doi.org/10.1109/i-PACT52855.2021.9696751>
4. Singgih, M.L., Zakiyyah, F.F. and Andrew (2024) 'Machine learning for predictive maintenance: A literature review', in *2024 7th International Conference on Vocational Education and Electrical Engineering (ICVEE 2024): Charting the Course of Artificial Technology in Sustainable Society*, pp. 250–256. Available at: <https://doi.org/10.1109/ICVEE63912.2024.10823713>
5. Zhang, N., Zhao, J., Ma, L., Kong, H. and Li, H., 2022. *Tool wear monitoring based on transfer learning and improved deep residual network*. IEEE Access. Available at: <https://ieeexplore.ieee.org/document/9941668>
6. Shao, H., Li, W., Cai, B., Wan, J., Xiao, Y. and Yan, S., 2023. *Uncertainty-aware deep learning: A promising tool for trustworthy fault diagnosis*. IEEE Transactions on Industrial Informatics. Available at: <https://www.ieee-jas.net/article/doi/10.1109/JAS.2024.124290>
7. Serradilla, O., Zugasti, E. and Zurutuza, U., 2020. *Deep learning models for predictive maintenance: A survey, comparison, challenges and prospect*. arXiv preprint arXiv:2010.03207. Available at: <https://arxiv.org/abs/2010.03207>
8. Samatas, G.G., Moumgiakmas, S.S. and Papakostas, G.A., 2021. *Predictive maintenance—Bridging artificial intelligence and IoT*. arXiv preprint arXiv:2103.11148. Available at: <https://arxiv.org/abs/2103.11148>
9. Zabin, M., Choi, H.J. and Uddin, J., 2022. *Hybrid deep transfer learning architecture for industrial fault diagnosis using Hilbert transform and DCNN–LSTM*. The Journal of Supercomputing. Available at: <https://link.springer.com/article/10.1007/s11227-022-04830-8>
10. Jiya, T., 2019. 'Ethical Implications of Predictive Risk Intelligence'. Centre for Computing and Social Responsibility, De Montfort University. *ORBIT Journal*, 2(2). Available at: <https://doi.org/10.29297/orbit.v2i2.112>
11. Syed, A., Manan, R.A., Ahmed, S., Shaikh, A. and Raza, B. (2020) 'A novel linear classification approach to deal with imbalanced class problem in failure prediction of air pressure systems', in *2020 3rd International Conference on Computing*,

- Mathematics and Engineering Technologies (iCoMET)*. IEEE, pp. 1–6. Available at: <https://doi.org/10.1109/iCoMET48670.2020.9073982>
12. Sridhar, N. and Sanagavarapu, S. (2021) 'A machine learning approach for predictive maintenance in industrial machinery using SMOTE for class imbalance', in *2021 International Conference on Artificial Intelligence and Smart Systems (ICAIS)*. IEEE, pp. 627–633. Available at: <https://doi.org/10.1109/ICAIS50930.2021.9395985>
  13. Susto, G.A., Beghi, A. and De Luca, C. (2015) 'A predictive maintenance system for industrial machines based on a multiple classifier approach', in *IEEE Transactions on Industrial Informatics*, 11(3), pp. 812–820. Available at: <https://doi.org/10.1109/TII.2014.2349359>
  14. Lorenti, L., Dalle Pezze, D., Andreoli, J., Masiero, C., Gentner, N., Yang, Y. and Susto, G.A. (2023) 'Predictive maintenance in the industry: A comparative study on deep learning-based remaining useful life estimation', in *Journal of Intelligent Manufacturing*. Springer. Available at: <https://doi.org/10.1007/s10845-023-02128-5>
  15. Hafeez, I., Naeem, M., Shafique, M., Anwar, S. and Ghafoor, M. (2021) 'Sequential multivariate fault prediction for vehicular predictive maintenance using machine learning', in *2021 International Conference on Emerging Trends in Smart Technologies (ICETST)*. IEEE, pp. 1–6. Available at: <https://doi.org/10.1109/ICETST52233.2021.9415942>